

國立雲林科技大學工業工程與管理系

碩士論文

Department of Industrial Engineering and Management

National Yunlin University of Science & Technology

Master Thesis

應用自動編碼器架構於影片生成文字敘述

- 以美國職業籃球聯賽比賽為例

Using an Autoencoder Framework to Generate Text

Narratives in Videos

- An Example of NBA Games



馬承安

Chang-An Ma

指導教授：陳維婷 博士

陳奕中 博士

Advisor: Wei-Ting Chen, Ph.D.

Yi-Chung Chen, Ph.D.

中華民國 112 年 6 月

June 2023

摘要

近年來，NBA 比賽的文字直播是一個新興生意，目前文字直播工作完全透過人工處理，造成文字直播員作業上的困難點，原因是場上球員的移動速度快，再加上每次進攻回合的時間不一致，導致文字直播內容容易出錯，使用者不信任網路平台服務，造成觀眾、廣告費收益減少。因此本研究希望能設計一種自動編碼器框架用於半自動化文字直播，以降低文字直播員的工作負荷量，而此框架的解題步驟包含編碼部分的串聯 CNN 和 YOLO 篩選目標畫面，以及解碼部分的(1)利用 Lightweight OpenPose 來找出投籃的球員，(2)利用 3D-CNN 進行得分種類辨識，以及(3)將各模型的輸出結果合併成文字敘述。最終我們的實驗結果則證實了我們所提出框架的有效性。

關鍵字：文字直播、YOLO-v5、Lightweight OpenPose、3D-CNN



Abstract

In recent years, the text live stream of NBA games is a new business. At present, the text live stream work is completely processed manually, which causes difficulties to the text live streamers. The reason is the players on the court moving too fast and each offensive round inconsistencies in time, leads to error-prone live text content. Resulting in reduced audience and advertising revenue. Therefore, this research proposes an automatic encoder framework for semi-automatic text live streaming to reduce the workload of text live streamers. The problem-solving steps of this framework includes the following steps: in the encoding part, connecting CNN and YOLO to filter the target screen, and in the decoding part: (1) Using Lightweight OpenPose for finding the shooting player, (2) Using 3D-CNN to identify the type of scoring, and (3) Combining the output results of each model into a text description. Finally, our experimental results ultimately confirm the effectiveness of the proposed framework.

Keywords : Text live stream, YOLO-v5, Lightweight OpenPose, 3D-CNN

目錄

摘要.....	2
Abstract.....	3
目錄.....	4
表目錄.....	6
圖目錄.....	8
第一章 緒論.....	11
1.1 研究背景與動機.....	11
1.2 研究目的.....	14
1.3 研究限制.....	15
1.4 研究流程.....	15
第二章 文獻回顧.....	17
2.1 物件偵測方法.....	17
2.1.1 兩階段偵測.....	17
2.1.2 單階段偵測.....	18
2.2 人體姿態辨識方法.....	18
2.2.1 Top-Down	19
2.2.2 Bottom-Up	20
2.3 得分種類辨識方法.....	21
2.4 文獻回顧小結.....	21
第三章 資料集.....	23
3.1 NBA 直播影片	23
3.2 NBA 文字資料	24
3.3 NBA 球員背號資料	25
第四章 研究方法.....	26
4.1 資料前處理.....	27
4.1.1 跳幀處理.....	27
4.1.2 角點偵測.....	28
4.1.3 將遠近鏡頭畫面進行分類.....	29
4.1.4 偵測球和籃框位置.....	30
4.1.5 辨識記分板內的時間.....	31
4.1.6 清洗文字資料.....	32
4.1.7 組合成訓練資料.....	33

4.2	訓練 Lightweight OpenPose 找出投籃的球員	33
4.3	辨識球員背號和球衣顏色.....	34
4.4	得分種類辨識.....	35
4.4.1	降低影像畫面大小、決定間格幀數.....	36
4.4.2	訓練單一模型辨識得分種類.....	37
4.4.3	訓練混合式模型辨識得分種類.....	38
4.4.4	訓練兩個模型辨識得分種類.....	40
4.5	各模型之評估指標.....	41
第五章	實驗模擬.....	44
5.1	資料集介紹.....	44
5.2	決定間格幀數大小.....	45
5.3	CNN 在將遠近鏡頭之比賽畫面進行分類的效果	46
5.4	YOLO-v5 在偵測畫面中球和籃框位置的效果	48
5.5	Lightweight OpenPose 辨識場上球員的效果.....	52
5.6	不同資料前處理方法對 3D-CNN 的預測效果.....	55
5.6.1	降低影像畫面大小、決定間格幀數.....	56
5.6.2	訓練單一模型辨識得分種類.....	58
5.6.3	訓練混合式模型辨識得分種類.....	59
5.6.4	訓練兩個模型辨識得分種類.....	61
5.7	將各模型預測結果合併成文字敘述的效果.....	63
5.8	模擬平行運算所需的電腦數量.....	64
第六章	結論與未來研究建議.....	66
參考文獻		67
附件一	NBA 比賽的記分板位置	72

表目錄

表 1 NBA 直播影片之相關資訊	23
表 2 清洗前的資料集	32
表 3 清洗後的資料集	33
表 4 混淆矩陣	41
表 5 COCO 資料集平台提供的每個關節點標準差(σ_i)[8]	43
表 6 實驗模擬所用到的直播影片	45
表 7 電腦在不同間格幀數下的處理時間	46
表 8 各類別在訓練、測試集的影像數量	47
表 9 CNN 的測試績效	47
表 10 YOLO 驗證集績效	50
表 11 YOLO 測試集績效	50
表 12 YOLOv5m 偵測球員投籃的過程	50
表 13 訓練 3D-CNN 的輸出類別數量	56
表 14 間格幀數為 20 幀、不同影像大小的 3D-CNN 績效	57
表 15 間格幀數為 30 幀、不同影像大小的 3D-CNN 績效	57
表 16 單一模型辨識得分種類的測試績效	58
表 17 測試 3D-CNN 之混淆矩陣	58
表 18 混合式模型辨識得分種類的測試績效	59
表 19 測試混合式 3D-CNN 之混淆矩陣	60
表 20 3D-CNN 1 的測試績效	62
表 21 3D-CNN 2 的測試績效	62
表 22 測試 3D-CNN 1 之混淆矩陣	62
表 23 測試 3D-CNN 2 之混淆矩陣	62
表 24 合併 3D-CNN 1 和 3D-CNN 2 測試結果之混淆矩陣	63

表 25 本研究框架生成文字敘述的效果.....	64
表 26 模擬電腦平行運算的過程.....	65



圖目錄

圖 1 文字直播示意圖.....	11
圖 2 自動編碼器框架的輸入和輸出之示意圖.....	12
圖 3 統計球員動作之出現次數(2015 ~ 2021 賽季).....	13
圖 4 非比賽畫面.....	13
圖 5 近鏡頭的比賽畫面.....	14
圖 6 遠鏡頭的比賽畫面.....	14
圖 7 研究流程圖.....	16
圖 8 物件偵測方法[5].....	17
圖 9 人體姿態辨識方法[27].....	19
圖 10 OpenPose 模型之學習框架[25].....	21
圖 11 訓練 OpenPose 模型之過程[3].....	21
圖 12 BASKETBALL REFERENCE 網站介面[1].....	24
圖 13 BASKETBALL REFERENCE 網站之文字資料[1].....	25
圖 14 查詢球員背號的 NBA 官網介面[24].....	25
圖 15 Offline 研究架構流程圖.....	26
圖 16 Online 研究架構流程圖.....	27
圖 17 角點偵測示意圖[44].....	28
圖 18 找出記分板內的比賽剩餘時間位置示意圖.....	29
圖 19 CNN 模型架構示意圖.....	29
圖 20 卷積層運算過程.....	30
圖 21 YOLO-v5 框架[26].....	31
圖 22 YOLO-v5 偵測球和籃框位置的過程.....	31
圖 23 辨識記分板內的時間.....	32
圖 24 訓練 Lightweight OpenPose 過程.....	34

圖 25 人體關節點位置.....	35
圖 26 找出投籃的球員之過程(示意圖).....	35
圖 27 訓練 3D-CNN 過程之示意圖	36
圖 28 3D-CNN 架構.....	37
圖 29 不同資料前處理下的影像.....	38
圖 30 壓縮影像前後的資料分布.....	38
圖 31 籃框下發球線的畫面.....	39
圖 32 混合式 3D-CNN 架構.....	39
圖 33 不同資料前處理下的影像(3D-CNN 2 輸入)	40
圖 34 不同資料前處理下的影像.....	40
圖 35 兩個 3D-CNN 之框架	41
圖 36 預測、實際邊界框之交集和並集.....	42
圖 37 間格幀數為 60 幀時的畫面.....	46
圖 38 遠近鏡頭的比賽畫面.....	48
圖 39 CNN 架構.....	48
圖 40 球被場上球員擋住的畫面.....	51
圖 41 球正在移動的畫面.....	51
圖 42 YOLOv5m 偵測多顆球的畫面	51
圖 43 球與籃框距離之視覺化.....	52
圖 44 球與籃框位置最近的畫面.....	52
圖 45 透過 Lightweight OpenPose 找出投籃球員的示意圖	54
圖 46 球與投籃球員手腕距離之視覺化.....	54
圖 47 投籃球員背號未面向鏡頭的畫面.....	54
圖 48 投籃球員被其它球員擋住的畫面.....	55
圖 49 辨識投籃球員背號的過程.....	55
圖 50 球員在上籃或灌籃時不小心衝出發球線外的畫面.....	60

圖 51 球員投籃後沒進，然後球出界的畫面.....	60
圖 52 模擬 10 台電腦平行運算後的視覺化圖.....	65



第一章 緒論

1.1 研究背景與動機

在社會經濟迅速成長之下，使得國民所得持續提高，人們擁有更多的時間及金錢參與運動活動，因而加速運動產業的蓬勃發展。至今為止，運動產業成為現今主流休閒娛樂產業之一。其中傳播媒介更是將運動產業推動到國際市場，球迷們可透過電視、手機應用程式等方式來觀賞賽事。而美國職業籃球聯賽(National Basketball Association，簡稱為NBA)，平均每場比賽的觀看次數為1.05億次[14][16]，是全球最多人觀看的籃球聯盟。然而 NBA 賽事主要在美國的晚間時段進行，對於不同時區的國家而言，會遇到兩大問題：(1)時差問題使得人們必須去上班或上課，無法在家觀看電視轉播，(2)透過手機應用程式觀看直播所需要的網路流量大，若工作或上課環境的網路訊號不佳，會使得影片訊號延遲、畫面解析度變差，因此許多人轉為觀看文字直播。至今各大網路平台如虎撲體育、騰訊體育和直播吧等平台開始經營文字直播的業務，直播過程為文字直播員在觀看比賽時，同時透過文字敘述將比賽資訊提供給球迷(如圖 1 所示)，其中比賽資訊會包含三大內容：比賽時間、賽事過程和雙方比分，使得球迷能即時得知比賽情形。至今為止，已超過 500 萬人利用文字直播觀看 NBA 賽事[43][45]。



圖 1 文字直播示意圖

但由於文字直播員須即時播報比賽內容，若無法掌握場上球員的移動速度以及每輪進攻回合的時間，在作業上會遇到兩大問題：(1)出錯機率高，(2)無法提供完整賽事資訊，進而導致使用者不信任文字直播的服務，使得網路平台的觀眾、廣告費收益減少。因此本研究希望能設計一種自動編碼器框架用於半自動化文字直播，透過輸入 NBA 直播影片來訓練模型，輸出則是由投籃的球員、投籃位置和是否得分組合而成的文字直播(圖 2)。而現階段僅考慮投籃項目(圖 3)，原因是過往文字轉播中，對於投籃描述比例佔約 7 成，換言之，若我們能自動化的從影片產生投籃的描述，即可降低文字直播員約 70% 的工作量，其餘 30% 的動作項目資訊則由文字直播員提供，能有效提升其他動作項目的文字直播品質。

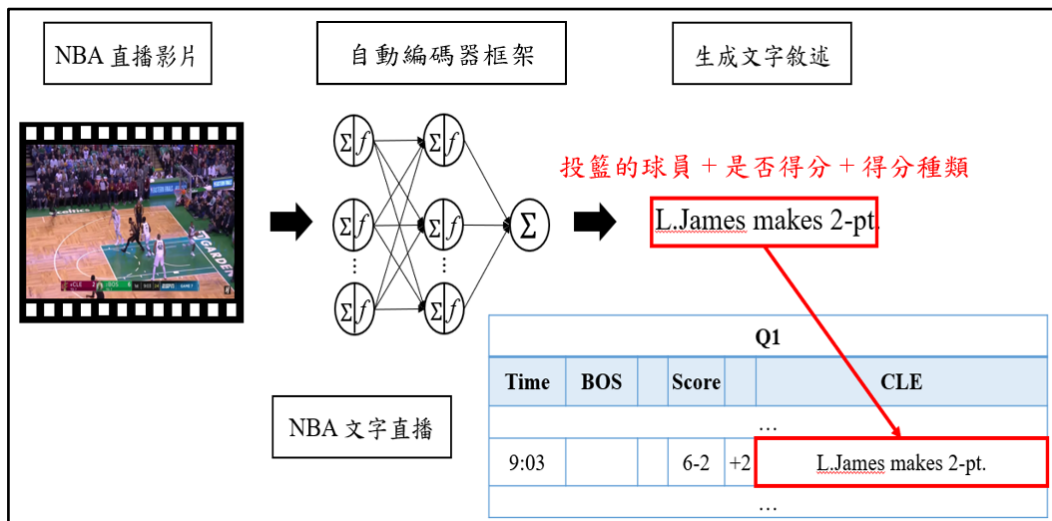


圖 2 自動編碼器框架的輸入和輸出之示意圖

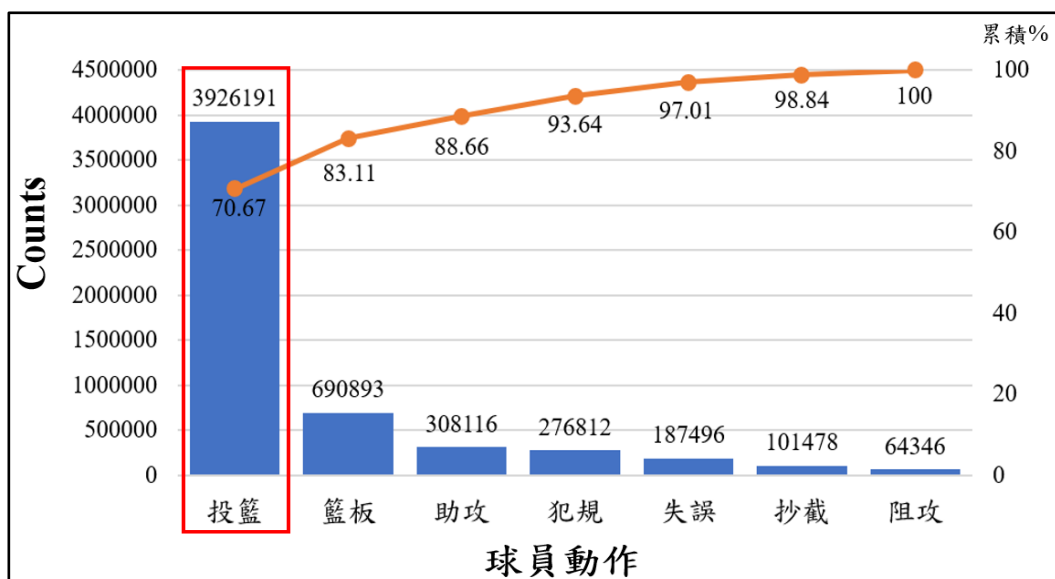


圖 3 統計球員動作之出現次數(2015 ~ 2021 賽季)

綜上所述，本研究框架僅考慮投籃項目，預分析的直播畫面就必須有投籃的球員、球和籃框，又稱為目標畫面，但是直播影片會包含其他畫面，分成非比賽畫面、近鏡頭的比賽畫面和遠鏡頭的比賽畫面(圖 4、5、6)。非比賽畫面的出現時機通常為球評評論、廣告推銷等，近鏡頭的比賽畫面則出現在重播回顧、場邊採訪、比賽暫停以及個人視角等，而遠鏡頭的比賽畫面是最主要的直播畫面，每個畫面都會包含投籃的球員、球和籃框，為本研究預分析的目標畫面。因此必須先篩選出目標畫面，再輸入至模型進行訓練。



圖 4 非比賽畫面



圖 5 近鏡頭的比賽畫面

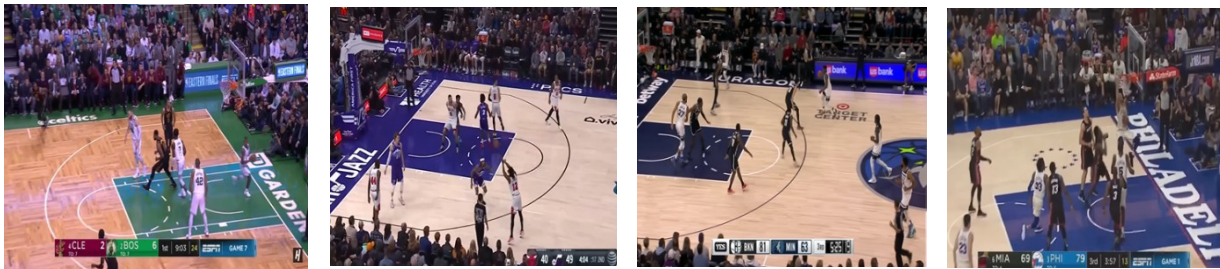


圖 6 遠鏡頭的比賽畫面

過往的研究方法應用在籃球賽事文字直播會有三大問題：(1)過往的篩選目標畫面模型必須搭配文字資料[6][15][28]，主要透過文字直播員所記錄的時間點，找出對應的目標畫面，但問題是直播當下並沒有文字資料，因此不適用於直播情形。(2)未探討投籃的球員是誰，Wu 等人[37]於 2019 年利用 CNN-LSTM 來辨識得分種類。Shakya 等人[31]在 2021 年則透過 3D-CNN 來辨識得分種類，上述研究能根據場上球員的得分種類進行分類，但都未能得知投籃的球員是誰。(3)未探討不同投籃位置下的得分種類，Yang 等人[38]在 2021 年為了幫助球員進行投籃訓練，將比賽影片分成得分以及未得分類別，並使用 HOG 結合 SVM 進行分類，但未探討球員在不同投籃位置(3 分線、2 分線和罰球線)的得分種類。

1.2 研究目的

為了改善並解決過往研究三大問題，本研究提出三大步驟來解題：(1)為解決過往的篩選目標畫面模型必須搭配文字資料問題，將篩選過程又分成四個部分，第一部分是將影片畫面進行角點偵測處理，第二部分則是訓練 CNN 將遠近鏡頭之比賽畫面進行分類，第三部分是訓練 YOLO-v5 偵測畫面中球和籃框的位置，第四部份為使用 OCR 辨識記分板內的時間，透過上述四個步驟篩選出目標畫面，其篩選過程又稱為編碼器架構(Encoder)。(2)為解決過往研究未探討投籃的球員是誰問題，使用 Lightweight OpenPose 找出投籃的球員後，再使用 OCR 辨識該球員的背號。(3)為解決過往研究未探討不同投籃位置下的得分種類問題，本研究訓練 3D-CNN 進行得分種類辨識，最終將各模型的輸出結果合併成文字敘述，也就是本研究之解碼器架構(Decoder)。

1.3 研究限制

研究限制分成三個部分：(1)僅考慮遠鏡頭的比賽畫面。(2)僅探討得分種類。未考慮其他動作項目如運球、傳球、抄截、阻攻、搶籃板、犯規、發生失誤等動作，也不考慮比賽暫停、球隊執行何種戰術、發生鬥毆事件以及球員受傷情形。(3)不探討球員的得分動作。如後仰跳投、低位單打、歐洲步等得分動作。

1.4 研究流程

本研究的架構流程圖，如圖 7 所示，共包含七個階段，以下將進行說明：

1. 確認研究問題

藉由探討文字直播員作業上的困難點，以及過往研究應用於籃球賽事文字直播之問題，提出一種能同時考慮到投籃的球員、得分種類和是否得分的框架，以進行 NBA 比賽文字直播。

2. 相關文獻回顧

本研究問題之相關文獻所使用到的方法可分成三大部分進行探討：物件偵測、人體姿態辨識以及得分種類辨識。物件偵測方法用於偵測球和籃框的位置，人體姿態辨識方法用於檢測人體關節點，找出投籃的球員，得分種類辨識方法則用來辨識得分種類。

3. 資料蒐集與整理

本研究蒐集的資料集為 NBA 直播影片、NBA 文字資料以及球員背號資料，其中 NBA 直播影片的資料來源於 YouTube 網站，NBA 文字資料的來源於 Basketball-Reference 網站，而球員背號資料則是從 NBA 官網進行下載。

4. 建立研究方法

本研究以資料分析流程訂定研究方法，首先將收集到的 NBA 直播影片進行篩選目標畫面，接著訓練 Lightweight OpenPose 模型找出投籃的球員，再來使用 3D-CNN 辨識得分種類，最終將各模型的輸出結果合併成文字敘述，以進行 NBA 比賽文字直播。

5. 實驗模擬與分析

本研究依照研究方法架構進行實驗模擬，並透過模擬驗證模型績效以及運算時間進

行參數修正與調整。

6. 結論與探討

根據實驗模擬後的模型之預測結果來驗證本研究框架的運算時間是否達到文字直播需求、是否能有效篩選目標畫面以及模型預測結果之可信度，並透過研究結果提出結論與未來相關之研究建議。

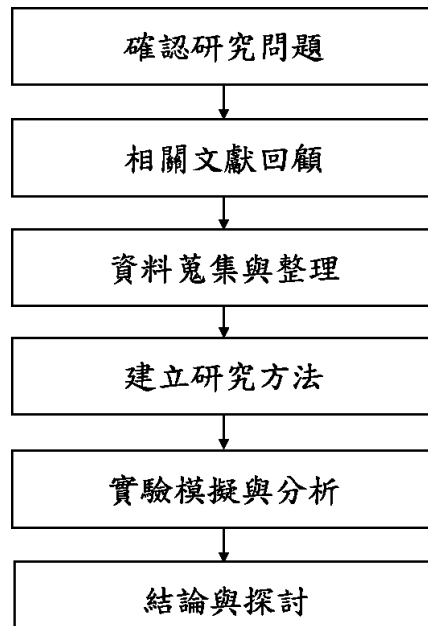


圖 7 研究流程圖

第二章 文獻回顧

本章節將介紹與本研究議題有關的文獻，以下將分別以四個小節進行說明，分別為物件偵測方法、人體姿態辨識方法、得分種類辨識方法以及文獻回顧小結。

2.1 物件偵測方法

物件偵測(Object Detection)為影像辨識領域的演算法之一，包含物件定位(Object Localization)以及物件辨識(Object Recognition)兩大概念，其作法是在圖像或影片內容中，透過邊界框標出物件範圍，再將此物件進行分類，最終給予此物件類別的猜測機率。如圖 8 所示，物件偵測方法又可區分成兩階段偵測(Two-stage)和單階段偵測(One-stage)，以下將針對這兩種方法進行說明。

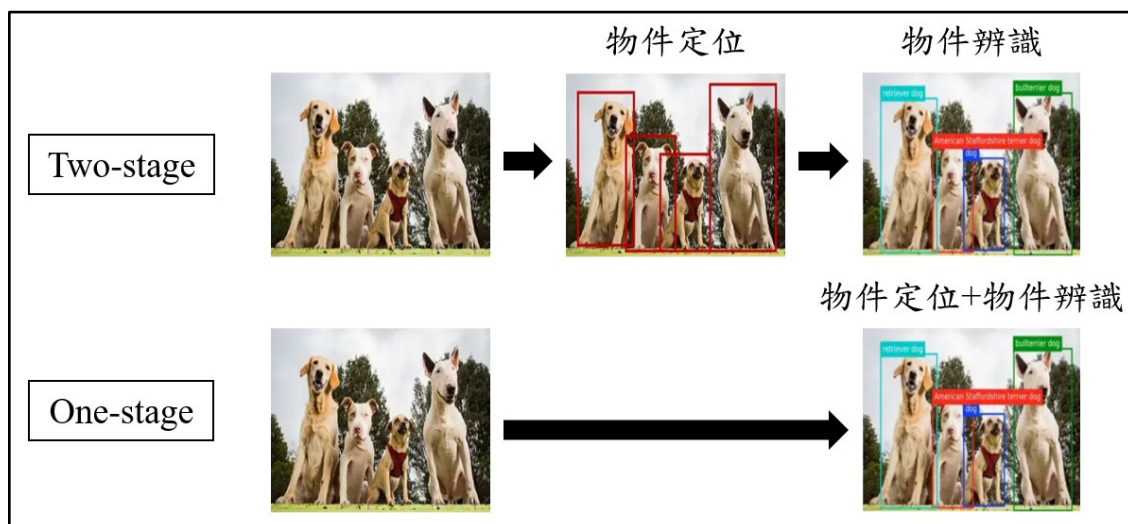


圖 8 物件偵測方法[5]

2.1.1 兩階段偵測

兩階段偵測方法主要會分成兩個步驟來偵測目標物件，第一個步驟是框出物件的位置，第二個步驟則進行物件辨識，最終偵測出該物件的位置以及類別，其中常見的兩階段偵測方法有 Fast R-CNN、Faster R-CNN 和 Mask R-CNN。Shi 等人[32]於 2017 年使用 Fast R-CNN 進行車輛檢測，Kim 等人[19]在 2020 年使用 Faster R-CNN 來辨識車輛類型；Zhang 等人[41]在 2017 年使用 Faster R-CNN 進行行人檢測；Liu 等人[22]在 2017 年分別使用 VGG16、ResNet101 和 PVANET 架構訓練 Faster R-CNN，並透過 mAP(mean Average Precision)來驗證模型績效，最終發現 PVANET 結合 Faster R-CNN 的效果最好，

mAP 為 85%；Li & Cheng[21]在 2019 年利用 Mask R-CNN 偵測行人性別；Buric 等人[2]於 2018 年使用 Mask R-CNN 和 YOLO 檢測球的位置，最終結果發現 Mask R-CNN 無法偵測到離鏡頭較遠的球，但是它的誤報率較低，因此辨識準確度還是比 YOLO 高。兩階段偵測方法在辨識物件的效果較佳，但由於偵測過程共有兩個步驟，導致運算時間較久。

2.1.2 單階段偵測

相較於兩階段偵測方法的偵測過程，單階段偵測方法僅透過一個步驟，即可辨識目標物件，換句話說，此方法同時進行物件定位以及物件辨識，辨識結果通常較兩階段偵測方法差，但運算時間較短，能達到即時辨識效果，常見的單階段偵測方法有 YOLO 和 SSD(Single Shot MultiBox Detector)。Yan 等人[38]在 2021 年使用 YOLO-v3 來辨識桌球選手的動作；Zhang 等人[42]在 2020 年將 YOLO-v4 結合 DeepSORT 演算法，來追蹤球員在場上的位置，其中最大 IoU(Intersection over Union)為 0.98，但當場上有兩名以上的球員重疊在鏡頭畫面時，容易有誤判情形；Fan 等人[10]於 2021 年使用 YOLO-v5、Faster R-CNN 和 EfficientDet 進行胸片異常檢測，辨識肺部是否有疾病徵兆，最終發現 YOLO-v5 的檢測效果最佳，辨識準確度比 Faster R-CNN 和 EfficientDet 高出 7.28%和 5.89%；Verma 等人[36]於 2021 年使用 YOLO-v3、YOLO-v4 和 YOLO-v5 對大豆作物進行昆蟲檢測，最終發現 YOLO-v5 擁有最佳的檢測精度，其 mAP 為 99.5%；Deepa 等人[9]在 2019 年使用 SSD 追蹤網球的軌跡；Jin 等人[17]在 2019 年使用 Faster R-CNN 和 SSD 偵測不同場景下的汽車位置，實驗結果發現 Faster R-CNN 具有更高的檢測效果，同時也能檢測出佔影片畫面較小的車輛，但有時會誤檢測出佔影片畫面較大的車輛，而 SSD 對佔影片畫面較小的車輛辨識準確度較低，但對佔影片畫面較大的車輛辨識準確度較高。

2.2 人體姿態辨識方法

人體姿態辨識(Human Body Posture Recognition)方法主要應用於動作辨識領域，其概念是透過偵測人體關節點來辨識動作，而每個關節點都有對應的平面座標，便於提取人體部位進行分析，其中如圖 9 所示，人體姿態辨識方法又可分成 Top-Down 和

Bottom-Up，以下將針對這兩種方法進行說明。

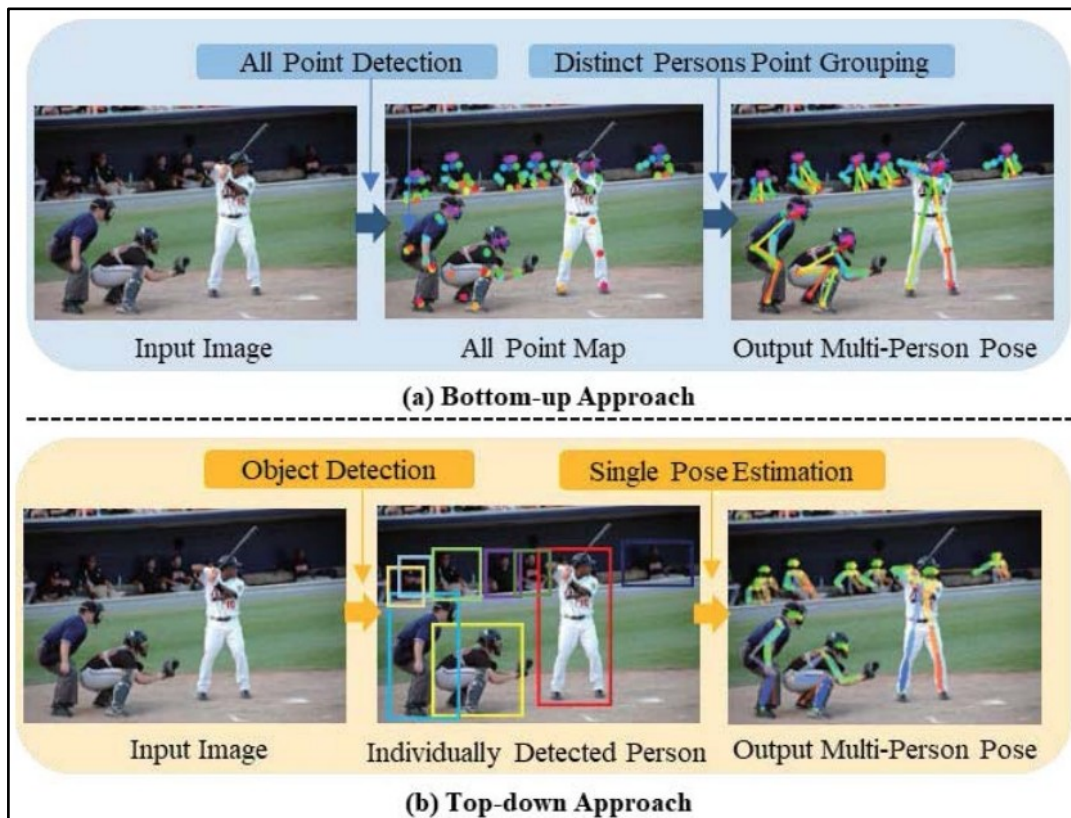


圖 9 人體姿態辨識方法[27]

2.2.1 Top-Down

如上所述，人體姿態辨識方法是透過偵測人體關節點來辨識動作，而 Top-Down 概念是先找出人體位置，以縮小偵測節點的範圍後，再來偵測人體關節點，雖然此方法的辨識效果佳，但模型執行時間會隨著影片中人數增加而拉長，常見的 Top-Down 方法有 Alpha-Pose、CPM 和 HRNet。Sun 等人[34]在 2022 年使用 CPM 檢測人體動作；Sun 等人[33]於 2019 年提出 HRNet 演算法，以進行人體姿勢辨識；Gamra 等人[12]於 2021 年使用 HRNet 檢測人體部位，最終結果顯示 HRNet 在檢測人體部位的 mAP 達到了 89.8%；Fang 等人[11]在 2017 年使用 Alpha-Pose 檢測多人情況下的姿態辨識，並且在實驗結果發現三大問題：(1)Alpha-Pose 無法辨識資料集中很少出現的姿勢，(2)當模型未找到人體位置，可能會遺漏檢測人體姿勢，(3)當某物體外觀與人類非常相似時，可能檢測到錯誤的姿勢。

2.2.2 Bottom-Up

Bottom-Up 概念是直接辨識影像中的人體關節點，省去物件定位的步驟，此作法的執行速度通常比 Top-Down 快，可用於即時辨識情形，常見的 Bottom-Up 方法有 OpenPose、Lightweight OpenPose 和 DensePose。Güler 等人[13]於 2018 年使用 DensePose 檢測人體動作；Cao 等人[3]在 2017 年提出 PAF(Point Affinity Fields)，來輔助 OpenPose 檢測多人情況下的姿態辨識(圖 11)；Choi 等人[7]在 2022 年為了訓練一個能辨識人體動作是否會跌倒的模型，於是先使用 YOLO-v4 檢測物件位置，再透過 OpenPose 提取跌倒和站立的人的特徵；Lv[23]在 2022 年使用 Lightweight OpenPose 檢測煤礦井下人員是否有危險行為，其中 Lightweight OpenPose 為輕量化的 OpenPose，兩個模型的差異在於(1)前者使用 MobileNet v1 取代 VGG-19 作為擷取骨幹特徵的模型，以達到輕量化模型的效果，(2) Lightweight OpenPose 部份的卷積層是使用空洞卷積(Dilation Convolution)來提升模型的學習視野，(3) Lightweight OpenPose 的卷積核尺寸為 3x3，後者是 7x7，(4)為了讓模型能更好的生成人體關節點熱力圖(Keypoint heatmaps)，Cao 等人[3]將 Openpose 的學習過程分成一個初始階段(Initial stage)和五個增強階段(Refinement stage)做訓練，而 Osokin 學者[25]則是將五個增強階段(Refinement stage)下降至一個增強階段(Refinement stage)來訓練 Lightweight OpenPose，以降低訓練模型的時間，(5) Cao 等人[3]採用兩個分支架構來分別學習人體關節點熱力圖(Keypoint heatmaps)和 PAF(Point Affinity Fields)，而 Osokin 學者[25]則是透過權值共享的方式，將兩個分支架構整合成一個，此作法可使模型參數量減少到 OpenPose 的 15%，辨識績效僅降低 1%。

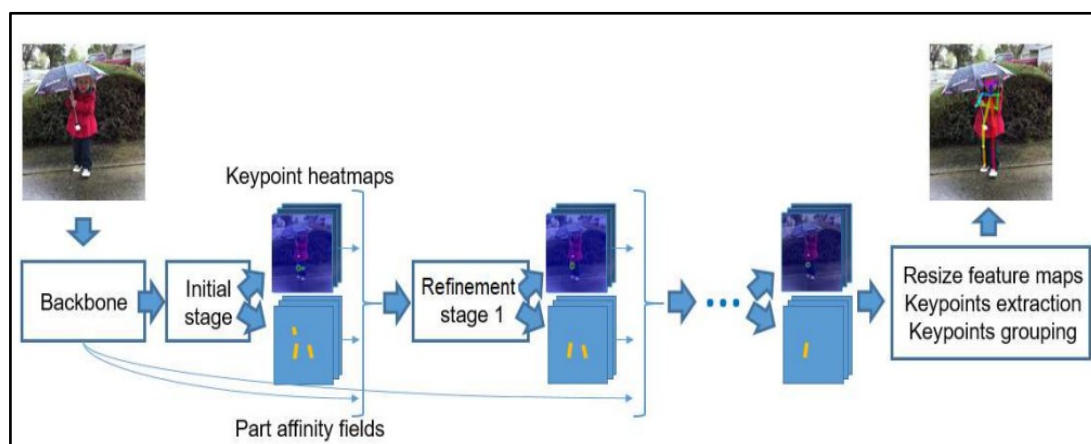


圖 10 OpenPose 模型之學習框架[25]

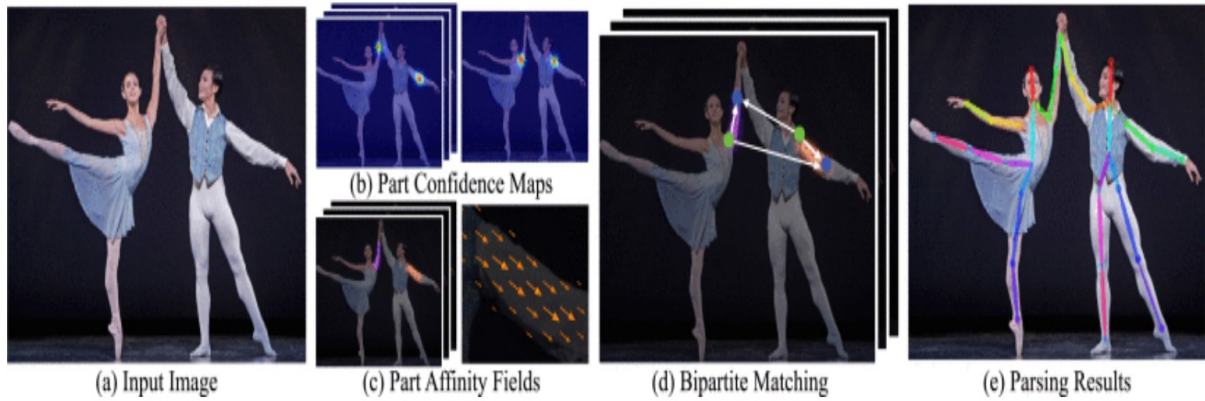


圖 11 訓練 OpenPose 模型之過程[3]

2.3 得分種類辨識方法

在運動賽事上，得分種類辨識可分成兩個分析議題：球員的投籃位置以及是否得分，而本小節將介紹過往根據球員的投籃位置或是否得分來進行分類之相關文獻。Chakma 等人[4]在 2020 年使用 CNN 預測足球選手的點球結果；Khan 等人[18]於 2018 年使用 C3D 檢測足球比賽的射門事件；Lee 等人[20]在 2018 年開發 DAF(Deterministic Finite Automata) 模組，透過追蹤球員位置以及動作辨識來識別籃球事件；Sen 等人[30]於 2022 年使用 DenseNet201、InceptionResNetV2、MobileNetV2、ResNet152V2 和 Xception 於影片中提取足球比賽的特徵，再透過 LSTM 進行動作分類，實驗結果顯示 DenseNet201 結合 LSTM 的分類效果最佳，F1-score 為 90%；Tegicho 等人[35]在 2021 年使用不同機器學習模型將罰球命中以及未命中進行分類，最終發現 SVM 的分類效果最佳，其準確率達到 88%。

2.4 文獻回顧小結

本研究將上述三小節統整成三大點做說明。(1)在物件偵測方法當中，兩階段偵測方法的辨識效果通常比單階段偵測方法佳，但是模型的運算時間比單階段偵測方法久，而本研究目標是要即時進行籃球賽事文字直播，因此單階段偵測方法較貼近本研究的需求，其中 YOLO-v5 是目前較常用來偵測物件的模型，不但運算速度快，也有良好的辨識效果，本研究將透過此模型偵測影片畫面中球和籃框的位置，並篩選出球位置與籃框最近的畫面。(2)人體姿態辨識方法是透過偵測人體關節點來進行動作辨識，而此方法可分成

Top-Down 和 Bottom-Up，其中 Bottom-Up 能直接偵測人體關節點位置，省去物件定位的步驟，並且達到即時辨識的效果。然而在 Bottom-Up 方法中，Lightweight OpenPose 的參數量較 OpenPose 少，不但能降低模型運算時間，又能保持良好的辨識效果，因此本研究將使用 Lightweight OpenPose 來找出投籃的球員。(3)過往應用在得分種類辨識的相關文獻大多是分別探討球員的投籃位置以及是否得分，而本研究想建立一個能同時考慮球員的投籃位置和是否得分的模型，以辨識球員的得分種類，其中 3D-CNN 能學習到影片中的時空關係，適合本研究在目標畫面中辨識得分種類。綜上所述，本研究使用 YOLO-v5、Lightweight OpenPose 建立投籃球員之辨識模型，再透過 3D-CNN 辨識得分種類，最終將不同模型的輸出結果合併成文字敘述。



第三章 資料集

本章介紹本實驗所使用到的兩種資料集，第一種類型為 NBA 直播影片，用於訓練模型以擷取影片中投籃的球員以及球的特徵；第二種類型為 NBA 文字資料，目標是提供給 Lightweight OpenPose 和 3D-CNN 作為輸出資料，以下分兩節進行資料說明。

3.1 NBA 直播影片

本研究所使用的資料集來源於 YouTube 網站[40]，總共收集 42 部直播影片，平均每部影片的長度為一小時半至兩個小時，畫面大小為 1280×720，解析度為 720p，其中有 28 部影片為 30fps (Frame per second)、14 部為 60fps(Frame per second)，總容量有 95GB，表 1 為本研究所下載的 NBA 直播影片相關資訊，其中日期欄位為比賽日期。

表 1 NBA 直播影片之相關資訊

影片編號	日期	對戰隊伍	影片長度	幀數(fps)
1	2010-05-27	洛杉磯湖人 vs 鳳凰城太陽	1:46:44	30
2	2012-02-19	奧克拉荷馬雷霆 vs 丹佛金塊	1:42:39	30
3	2012-04-08	紐約尼克 vs 芝加哥公牛	2:09:38	30
4	2012-05-19	奧克拉荷馬雷霆 vs 邁阿密熱火	1:34:00	30
5	2012-05-30	波士頓賽爾蒂克 vs 邁阿密熱火	2:04:46	30
6	2013-01-17	洛杉磯湖人 vs 邁阿密熱火	1:47:56	30
7	2013-02-27	金洲勇士 vs 紐約尼克	1:33:32	30
8	2013-04-21	奧克拉荷馬雷霆 vs 休士頓火箭	1:41:50	30
9	2013-05-06	金洲勇士 vs 聖安東尼奧馬刺	2:08:41	30
10	2013-05-24	印第安那溜馬 vs 邁阿密熱火	1:57:51	30
11	2014-05-14	邁阿密熱火 vs 布魯克林籃網	1:41:06	30
12	2013-06-01	印第安那溜馬 vs 邁阿密熱火	2:16:26	30
...	...	...	...	...
42	2022-04-02	達拉斯獨行俠 vs 費城 76 人	1:47:55	60

3.2 NBA 文字資料

本研究所收集的 NBA 文字資料，來自 BASKETBALL REFERENCE 網站[1]，圖 12 為網站介面，使用者能在此網站透過預查詢的 NBA 賽季、比賽時間、特定球隊以及比賽地點等資訊，取得當天比賽文字直播員所紀錄的文字資料。而圖 13 為文字資料的介面，畫面為布魯克林籃網隊(Brooklyn Nets)和密爾瓦基公鹿隊(Milwaukee Bucks)的比賽，資料欄位包含節數(Quarter)、時間(Time)、主場球隊(Home Team)、客場球隊(Away Team)、雙方比分(Score)以及此輪進攻回合主場、客場球隊得分紀錄，其中在主、客場球隊欄位主要是記錄籃球事件。

2021-22 NBA Season

Standings

Schedule and Results

Leaders

Coaches

Player Stats

Other

2022 Playoffs Summary

Back to top

October

November

December

January

February

March

April

May

June

October Schedule

Share & Export

Date	Start (ET)	Visitor/Neutral	PTS	Home/Neutral	PTS	Attend.	Arena	Notes
Tue, Oct 19, 2021	7:30p	Brooklyn Nets	104	Milwaukee Bucks	127	Box Score	17,341 Fiserv Forum	
Tue, Oct 19, 2021	10:00p	Golden State Warriors	121	Los Angeles Lakers	114	Box Score	18,997 Crypto.com Arena	
Wed, Oct 20, 2021	7:00p	Indiana Pacers	122	Charlotte Hornets	123	Box Score	15,521 Spectrum Center	
Wed, Oct 20, 2021	7:00p	Chicago Bulls	94	Detroit Pistons	88	Box Score	20,088 Little Caesars Arena	
Wed, Oct 20, 2021	7:30p	Boston Celtics	134	New York Knicks	138	Box Score	20T 19,812 Madison Square Garden (IV)	
Wed, Oct 20, 2021	7:30p	Washington Wizards	98	Toronto Raptors	83	Box Score	19,800 Scotiabank Arena	
Wed, Oct 20, 2021	8:00p	Cleveland Cavaliers	121	Memphis Grizzlies	132	Box Score	15,975 FedEx Forum	
Wed, Oct 20, 2021	8:00p	Houston Rockets	106	Minnesota Timberwolves	124	Box Score	16,079 Target Center	
Wed, Oct 20, 2021	8:00p	Philadelphia 76ers	117	New Orleans Pelicans	97	Box Score	12,845 Smoothie King Center	
Wed, Oct 20, 2021	8:30p	Orlando Magic	97	San Antonio Spurs	123	Box Score	16,697 AT&T Center	
Wed, Oct 20, 2021	9:00p	Oklahoma City Thunder	86	Utah Jazz	107	Box Score	18,306 Vivint Smart Home Arena	
Wed, Oct 20, 2021	10:00p	Sacramento Kings	124	Portland Trail Blazers	121	Box Score	17,467 Moda Center	
Wed, Oct 20, 2021	10:00p	Denver Nuggets	110	Phoenix Suns	98	Box Score	16,074 Phoenix Suns Arena	
Thu, Oct 21, 2021	7:30p	Dallas Mavericks	87	Atlanta Hawks	113	Box Score	17,162 State Farm Arena	
Thu, Oct 21, 2021	8:00p	Milwaukee Bucks	95	Miami Heat	137	Box Score	19,600 FTX Arena	
Thu, Oct 21, 2021	10:00p	Los Angeles Clippers	113	Golden State Warriors	115	Box Score	18,064 Chase Center	
Fri, Oct 22, 2021	7:00p	New York Knicks	121	Orlando Magic	96	Box Score	18,846 Amway Center	
Fri, Oct 22, 2021	7:00p	Indiana Pacers	134	Washington Wizards	135	Box Score	OT 15,407 Capital One Arena	
Fri, Oct 22, 2021	7:00p	Charlotte Hornets	123	Cleveland Cavaliers	112	Box Score	17,116 Rocket Mortgage Fieldhouse	
Fri, Oct 22, 2021	7:30p	Toronto Raptors	115	Boston Celtics	83	Box Score	19,156 TD Garden	
Fri, Oct 22, 2021	7:30p	Brooklyn Nets	114	Philadelphia 76ers	109	Box Score	20,367 Wells Fargo Center	
Fri, Oct 22, 2021	8:00p	Oklahoma City Thunder	91	Houston Rockets	124	Box Score	15,674 Toyota Center	
Fri, Oct 22, 2021	8:00p	New Orleans Pelicans	112	Chicago Bulls	128	Box Score	20,995 United Center	
Fri, Oct 22, 2021	9:00p	San Antonio Spurs	96	Denver Nuggets	102	Box Score	19,520 Ball Arena	
Fri, Oct 22, 2021	10:00p	Phoenix Suns	115	Los Angeles Lakers	105	Box Score	18,997 Crypto.com Arena	
Fri, Oct 22, 2021	10:00p	Utah Jazz	110	Sacramento Kings	101	Box Score	17,583 Golden 1 Center	
Sat, Oct 23, 2021	6:00p	Atlanta Hawks	95	Cleveland Cavaliers	101	Box Score	16,846 Rocket Mortgage Fieldhouse	
Sat, Oct 23, 2021	7:00p	Miami Heat	91	Indiana Pacers	102	Box Score	OT 17,147 Gainbridge Fieldhouse	
Sat, Oct 23, 2021	7:30p	Dallas Mavericks	103	Toronto Raptors	95	Box Score	19,800 Scotiabank Arena	
Sat, Oct 23, 2021	8:00p	Detroit Pistons	82	Chicago Bulls	97	Box Score	18,888 United Center	
Sat, Oct 23, 2021	8:00p	New Orleans Pelicans	89	Minnesota Timberwolves	96	Box Score	15,343 Target Center	
Sat, Oct 23, 2021	8:30p	Milwaukee Bucks	121	San Antonio Spurs	111	Box Score	14,353 AT&T Center	

圖 12 BASKETBALL REFERENCE 網站介面[1]

1st Q				
Time	Brooklyn	Score	Milwaukee	
12:00.0	Jump ball: N. Claxton vs. B. Lopez (G. Antetokounmpo gains possession)			
11:42.0		0-0	G. Allen misses 3-pt jump shot from 27 ft	
11:39.0	Defensive rebound by K. Durant	0-0		
11:27.0	Shooting foul by G. Antetokounmpo (drawn by N. Claxton)	0-0		
11:27.0	N. Claxton misses free throw 1 of 2	0-0		
11:27.0	Offensive rebound by Team	0-0		
11:27.0	N. Claxton misses free throw 2 of 2	0-0		
11:25.0		0-0	Defensive rebound by G. Antetokounmpo	
11:13.0		0-0	G. Antetokounmpo misses 3-pt jump shot from 26 ft	
11:10.0		0-0	Offensive rebound by B. Lopez	
11:01.0		0-0	G. Antetokounmpo misses 2-pt jump shot from 15 ft	
10:59.0	Defensive rebound by B. Griffin	0-0		
10:47.0	N. Claxton makes 2-pt dunk at rim (assist by J. Harden)	+2 2-0		
10:31.0		2-0	G. Antetokounmpo misses 2-pt hook shot from 11 ft	
10:29.0		2-0	Offensive rebound by G. Antetokounmpo	
10:28.0		2-2	+2	G. Antetokounmpo makes 2-pt layup from 1 ft
10:16.0	J. Harden misses 3-pt jump shot from 27 ft	2-2		
10:12.0		2-2	Defensive rebound by G. Antetokounmpo	
10:03.0		2-5	+3	B. Lopez makes 3-pt jump shot from 28 ft (assist by G. Antetokounmpo)
9:42.0	K. Durant makes 2-pt jump shot from 13 ft (assist by J. Harden)	+2 4-5		
9:32.0		4-5	G. Antetokounmpo misses 2-pt layup from 4 ft	
9:30.0		4-5	Offensive rebound by G. Antetokounmpo	
9:30.0		4-5	G. Antetokounmpo misses 2-pt layup from 2 ft (block by J. Harden)	
9:29.0		4-5	Offensive rebound by B. Lopez	
9:28.0		4-5	B. Lopez misses 2-pt hook shot from 5 ft	

圖 13 BASKETBALL REFERENCE 網站之文字資料[1]

3.3 NBA 球員背號資料

本研究所收集的 NBA 球員背號資料，來源於 NBA 官網[24]，圖 14 為該網站的介面，使用者能在此網站查詢每個球員的背號、所屬球隊、場上位置以及該球員的個人資訊，而本研究僅會用到兩個資料欄位，分別是球員背號以及所屬球隊。

PLAYER	TEAM	NUMBER	POSITION	HEIGHT	WEIGHT	LAST ATTENDED	COUNTRY
Precious Achiuwa	TOR	5	F	6-6	225 lbs	Memphis	Nigeria
Steven Adams	MEM	4	C	6-11	265 lbs	Pittsburgh	New Zealand
Bam Adebayo	MIA	13	C-F	6-9	255 lbs	Kentucky	USA
Ochai Agbaji	UTA	30	G	6-5	215 lbs	Kansas	USA
Santi Aldama	MEM	7	F-C	7-0	215 lbs	Loyola-Maryland	Spain
Nickail Alexander-Walker	MIN	9	G	6-5	205 lbs	Virginia Tech	Canada
Grayson Allen	MIL	12	G	6-4	198 lbs	Duke	USA
Jarrett Allen	CLE	31	C	6-9	243 lbs	Texas	USA
Jose Alvarado	NOP	15	G	6-0	179 lbs	Georgia Tech	USA
Kyle Anderson	MIN	5	F-G	6-9	230 lbs	UCLA	USA
Giannis Antetokounmpo	MIL	34	F	7-0	243 lbs	Filathioskos	Greece
Thanasis Antetokounmpo	MIL	43	F	6-7	219 lbs	Panathinaikos	Greece
Cole Anthony	ORL	50	G	6-3	185 lbs	North Carolina	USA

圖 14 查詢球員背號的 NBA 官網介面[24]

第四章 研究方法

本研究設計一種自動編碼器框架應用於半自動化文字直播 NBA 賽事。以下將介紹本研究方法，研究架構流程圖如圖 15、16 所示，共分為 Offline 訓練和 Online 預測。Offline 為訓練不同模型的過程，主要分成兩階段進行，第一階段為資料前處理，分為七大部分，包含(1)跳幀處理、(2)角點偵測、(3)將遠近鏡頭畫面進行分類、(4)偵測球和籃框位置、(5)辨識記分板內的時間、(6)清洗文字資料、(7)組合成訓練資料。透過第一階段來篩選目標畫面，以提升後續模型的預測績效。第二階段為訓練投籃球員之辨識模型以及得分分類模型。由於進行比賽時，籃球場上會出現 10 名球員，因此必須透過投籃球員之辨識模型找出投籃的球員，再使用得分分類模型辨識該球員的投籃位置以及是否進球，最終將這些模型的輸出結果合併成文字敘述。訓練完模型後，透過 Online 預測來進行 NBA 賽事的文字直播。

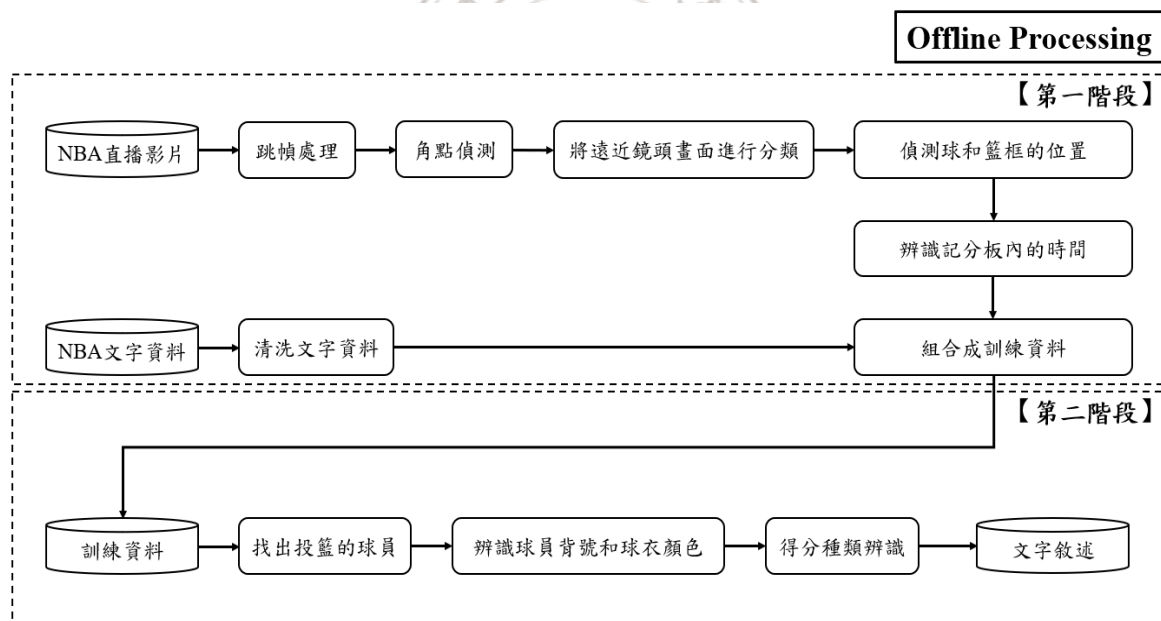


圖 15 Offline 研究架構流程圖

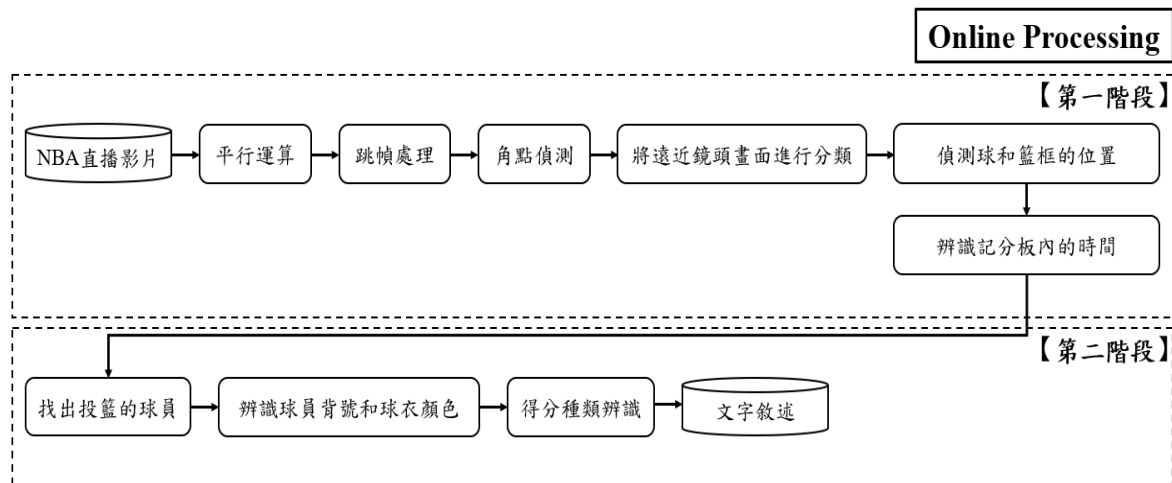


圖 16 Online 研究架構流程圖

4.1 資料前處理

資料前處理的重要性在於將資料轉換成便於模型訓練的格式，適當的資料前處理方式助於提升模型訓練績效，其中本研究收集的 NBA 直播影片會包含多個畫面，分別為非比賽畫面、近鏡頭的比賽畫面以及遠鏡頭的比賽畫面，而本研究預分析的目標畫面為遠鏡頭的比賽畫面，因此以下會介紹本研究所使用到的資料前處理過程，分別為(1)跳幀處理、(2)角點偵測、(3)將遠近鏡頭畫面進行分類、(4)偵測球和籃框的位置、(5)辨識記分板內的時間、(6)清洗文字資料、(7)組合成訓練資料七個部分，透過資料前處理篩選目標畫面。

4.1.1 跳幀處理

本研究所收集的 NBA 直播影片之幀數分別為 30fps 以及 60fps，代表一秒畫面會有 30 張或 60 張影像，而一場比賽的平均時間為一小時半至兩個小時，合計有超過 162,000 張影像，若將全部的影像輸入至電腦進行處理，不但會增加模型運算的負荷量，也會拉長電腦的處理時間，為了降低模型訓練量以及減少電腦的處理時間，本研究設計間格幀數來擷取影像，雖然此作法會遺失大量的影像，但是會減少電腦的處理時間，因此本研究會設計多個參數來做實驗，期望找出最佳參數，能在維持本研究框架的預測效果之下，又能降低電腦的處理時間。

4.1.2 角點偵測

角點偵測(Corner Detection)是電腦視覺(Computer Vision)領域中獲取影像特徵的一種方法，用來尋找影像中不同區域的交點，被廣泛應用於運動檢測、物件追蹤以及目標識別等議題，其概念是給定一個固定的搜索窗口，在灰階影像中的任意方向進行滑動，找出像素數值變化較大的特徵點，將它視為角點。如圖 17 所示，畫面中有多個不同形狀的積木，而紅點為角點偵測器所找出的角點，可以發現所有的角點都位於不同邊線的交點，以此就能繪製該物體的輪廓來識別目標，而本研究會使用目前較多學者使用的 Shi-Tomasi 角點偵測器，來找出記分板內的比賽剩餘時間位置(如圖 18 所示)，此方法的核心概念為尋找整張影像內最重要的前幾名特徵點，藉以提供更高質量的角點座標，並且能加速搜索的過程，而本研究會先透過資料觀察，將每場比賽的記分板位置記錄下來(如附件一所示)，其用意是降低角點偵測的搜索範圍，更聚焦在記分板內的比賽剩餘時間位置以提升搜索效率，總而言之，本研究希望透過此方法來排除非比賽畫面，僅留下近鏡頭以及遠鏡頭的比賽畫面，也便於 OCR(Optical Character Recognition)辨識時間資訊。

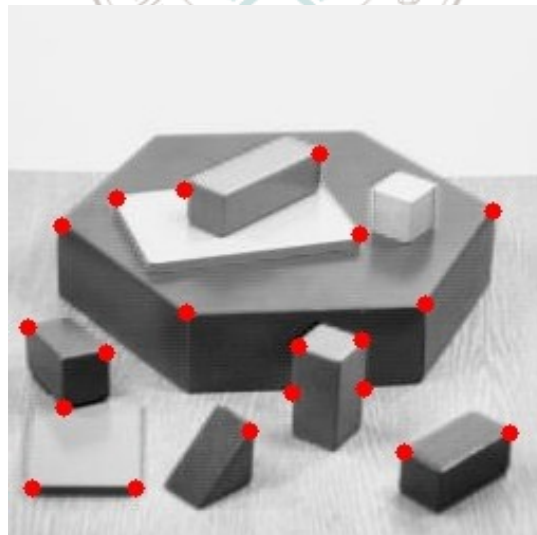


圖 17 角點偵測示意圖[44]

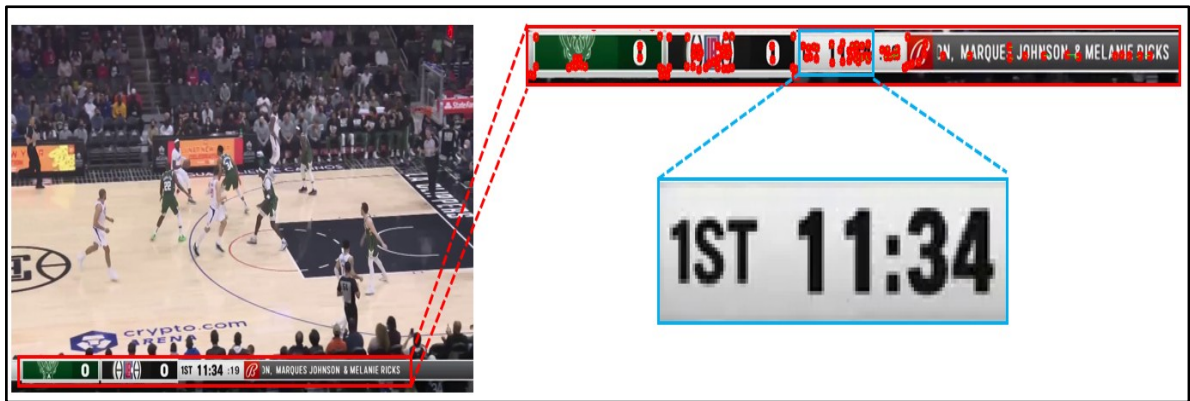


圖 18 找出記分板內的比賽剩餘時間位置示意圖

4.1.3 將遠近鏡頭畫面進行分類

本研究透過 CNN(Convolutional Neural Network)將遠近鏡頭畫面進行分類(圖 19)，篩選出遠鏡頭的比賽畫面，而 CNN 模型架構包含(1)輸入層、(2)卷積層、(3)池化層、(4)全連接層和(5)輸出層，以下將分別進行介紹。

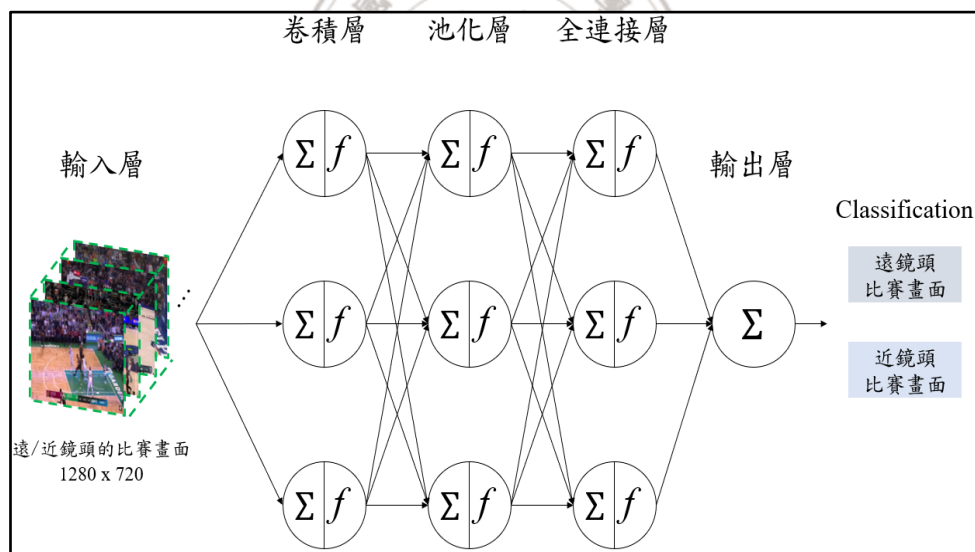


圖 19 CNN 模型架構示意圖

(1)輸入層：輸入層的工作就是將資料輸入至模型，但不進行任何計算，並以原始數值輸出至下一層，假設遠近鏡頭畫面大小為 3×3 ，則模型會以 3×3 的輸入維度進入下一層訓練。

(2)卷積層(Convolution Layer)：卷積層的任務就是要從遠近鏡頭畫面中擷取重要的空間資訊，其運作過程如圖 20 所示，假設模型學習的遠近鏡頭畫面大小為 5×5 ，卷積

核(Kernel)的大小為 3×3 ，透過卷積核在遠近鏡頭畫面上平滑移動過程，將網格內數值相乘後再相加算出數值，最終會得出一個新的矩陣進入下一層，目的就是讓模型能更有效的學習到空間資訊。

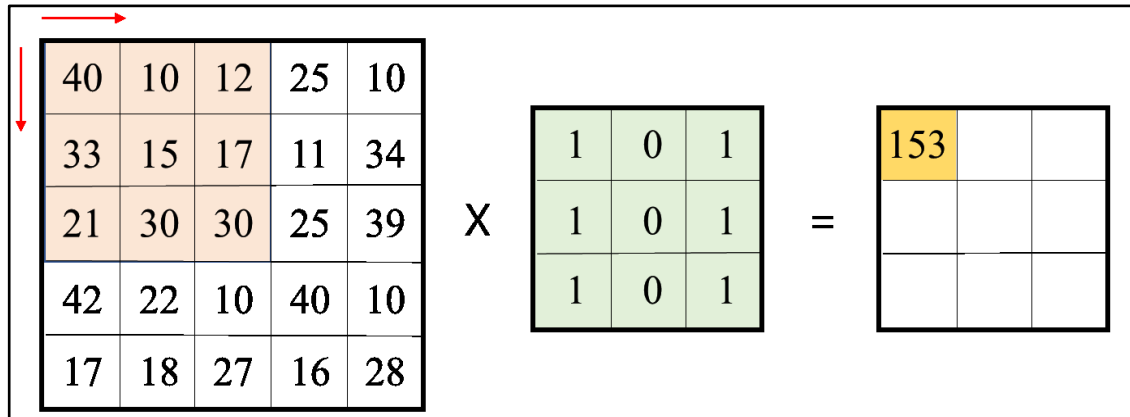


圖 20 卷積層運算過程

(3)池化層(Pooling Layer)：池化層則是將卷積層輸出的矩陣進行降維，目的是要突顯遠近鏡頭畫面的重要特徵區域，並且減少模型的參數量以提升運算速度，然而過往常見的有最大池化(Max Pooling)以及平均池化(Average Pooling)兩種卷積方法，這兩種卷積方法擷取空間特徵的方式不一致，最大池化法是找出網格內最大的數值，通常應用於分類議題，而平均池化法則是將網格內數值平均化，應用於影像色階處理，而本研究會採用最大池化法來進行池化動作。

(4)全連接層(Fully Layer)：目的是讓模型能學習到前面網路中的重要特徵，並且各神經元權重相同具有共享性，但也會因為各神經元權重彼此相連，進而產生出大量的參數，而本研究將透過全連接層計算卷積和池化後的結果並進行分類。

(5)輸出層：輸出兩個類別，分別為遠、近鏡頭的比賽畫面，本研究僅留下遠鏡頭的比賽畫面進行分析。

4.1.4 偵測球和籃框位置

本研究透過 YOLO-v5 偵測球和籃框的位置，目的是要找出球與籃框最近的畫面，代表球員剛完成投籃動作。如圖 21 所示，YOLO-v5 可分成 Backbone、Neck 和 Head 三大框架，Backbone 用於提取圖像中的重要特徵；Neck 用於生成特徵金字塔(Feature Pyramids)，有助於縮放特徵圖尺寸；最終由 Head 輸出此物件的類別機率以及邊界框位

置。圖 22 為 YOLO-v5 偵測球和籃框位置的過程示意圖，以下進行詳細說明，(1)輸入遠鏡頭的比賽畫面訓練 YOLO-v5。(2)比賽畫面會被切割成 $N*N$ 個網格，且每個網格大小一致。(3)球和籃框都會有一個邊界框(Bounding Box)，而每一個 Bounding Box 包含 5 個參數和此物件的類別，其中參數分別為中心點 x 、 y 座標、邊界框寬、高以及目標偵測的信心分數。(4)判斷物件中心點落在哪個網格，該網格就要負責偵測此物件。(5)反覆訓練模型以修正邊界框。(6)找到球與籃框在畫面中的位置。

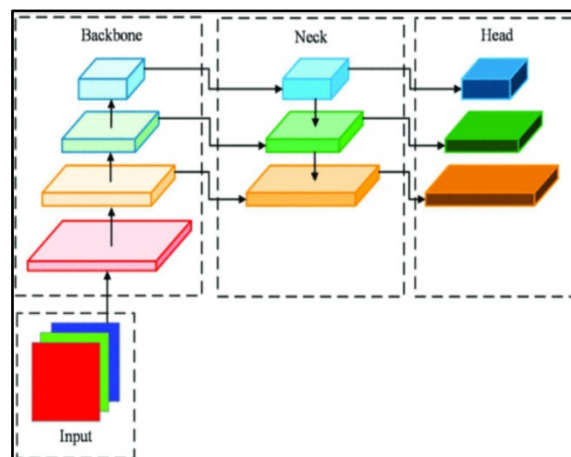


圖 21 YOLO-v5 框架[26]

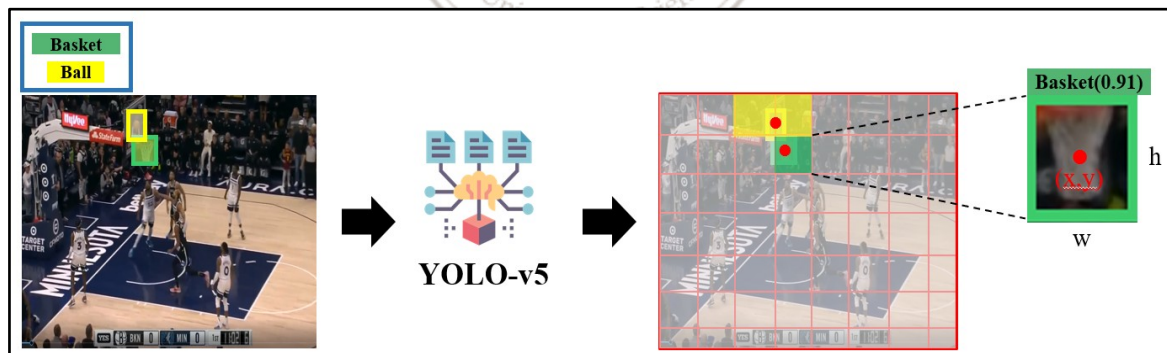


圖 22 YOLO-v5 偵測球和籃框位置的過程

4.1.5 辨識記分板內的時間

透過第 4.1.2 小節找出記分板內的比賽剩餘時間位置後，本小節會使用過往常被應用在偵測影像中文字的 OCR(Optical Character Recognition)模組，來辨識記分板內的時間資訊，如圖 23 所示，本研究之辨識目標包含節和比賽時間，並且將它們記錄下來，其中紅色框內容為第一節、橘色框則是比賽剩餘的時間。



圖 23 辨識記分板內的時間

4.1.6 清洗文字資料

本小節會將 NBA 文字資料進行清洗動作，其中表 2 為密爾瓦基公鹿隊和洛杉磯快艇隊於 2022 年 2 月 6 號的文字直播資料，可以看出原始資料有紀錄雙方球員在每回合所發生的事件，包含開場跳球、搶籃板以及投籃等動作，總共有 465 個事件，而本研究現階段僅考慮球員投籃時的情形，因此必須將投籃以外的事件進行刪除；再來為了建立訓練 3D-CNN 的輸出類別，本研究會合併球員的投籃位置和是否得分為類別，舉例來說，假設 A 球員在三分線出手未命中，本研究就會將「三分線」和「未命中」合併成 3pt_0，因此總共有 3pt_0、3pt_1、2pt_0、2pt_1、1pt_0 和 1pt_1 六個類別，再將這些類別也記錄到投籃位置+是否得分欄位；最後是將該球員的名字轉換成他的所屬球隊以及背號並輸入到投籃的球員欄位，便於 OCR 辨識數字，而清洗後的資料集如表 3 所示，僅剩下 182 個事件。

表 2 清洗前的資料集

Quarter	Time	Events
1	12:00	Jump ball: B. Portis vs. I. Zubac
1	11:39	J. Holiday misses 3-pt jump shot from 25 ft
1	11:35	Defensive rebound by N. Batum
1	11:29	R. Jackson makes 2-pt jump shot from 21 ft
1	11:15	K. Middleton misses 3-pt jump shot from 27 ft
1	11:12	Defensive rebound by M. Morris
1	11:09	Turnover by M. Morris
1	11:06	Shooting foul by M. Morris
...	...	...
4	00:00	End of 4th quarter

表 3 清洗後的資料集

Quarter	Time	投籃的球員	投籃位置 + 是否得分
1	11:39	公鹿隊(21 號)	3pt_0
1	11:29	快艇隊(1 號)	2pt_1
1	11:15	公鹿隊(22 號)	3pt_0
1	11:06	公鹿隊(34 號)	1pt_1
1	11:06	公鹿隊(34 號)	1pt_0
1	10:52	快艇隊(40 號)	2pt_0
1	10:44	公鹿隊(34 號)	2pt_1
1	10:20	快艇隊(8 號)	2pt_1
...	...	...	...
4	00:17	快艇隊(4 號)	2pt_1

4.1.7 組合成訓練資料

最終將目標畫面作為輸入資料，類別資料中的投籃位置+是否得分欄位作為輸出資料，並且合併成訓練資料來訓練 3D-CNN。

4.2 訓練 Lightweight OpenPose 找出投籃的球員

本章節將訓練 Lightweight OpenPose 找出投籃的球員，圖 24 為此模型的訓練過程，首先透過 MobileNet v1 提取人體關節點特徵，輸出的每張特徵圖都有一個關節點，接著為了避免訓練模型時，模型將不同人的關節點視為同一個人並連接起來，本研究透過 PAF(Point Affinity Fields)連接關節點至正確的人體位置，其作法如下，假設要將影像中每個人的手臂位置連接起來，首先要透過人體關節點熱力圖找出節點 6、7，也就是每個人的左肩膀($X_{j1,k}$)和左手肘($X_{j2,k}$)位置，找出這兩個節點後，即可計算肢體寬度(σ_l)和肢體長度($l_{c,k}$)，也就能形成一個手臂的範圍，再來會給定 P 點位置，若 P 點在手臂範圍內，就會產生一組向量(v)，否則為不會產生向量，若提供多個 P 點，且都位於手臂之間，就會產生多組向量在手臂位置上，也就能將兩個關節點連接起來，計算方式如公式 1、2、

3。解決上述問題後，仍然會遇到一個問題是，每個人的關節點具有順序關係，若未事先讓模型學習到人體關節點的順序關係，其預測的關節點位置會和真實情形差異大，因此本研究使用 Bipartite Matching 方法，先將不同人但相同的關節點分成同一群，並依照順序關係挑選任兩群關節點，並透過二分匹配找出最大流量的關節點並連接起來，就能完成人體姿態辨識。其中如圖 25 所示，每個關節點都會有對應的編號順序，詳細的關節點以及編號資訊如表 5 所示，而本研究將抓取不同球員的手腕位置，也就是節點 5 和 8 進行分析，目的是為了找出球與哪位球員的節點 5 或 8 位置最相近，即可判斷該球員就是投籃的球員(圖 26(a))。

$$L_{c,k}(P) = \begin{cases} v & \text{if } P \text{ on limb } c, k \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

$$v = \frac{(X_{j2,k} - X_{j1,k})}{\|X_{j2,k} - X_{j1,k}\|^2} \quad (2)$$

$$0 \leq v \cdot (P - x_{j1,k}) \leq l_{c,k} \text{ and } |v_{\perp} \cdot (P - x_{j1,k})| \leq \sigma_l \quad (3)$$

v 為肢體向量， $X_{jn,k}$ 代表第 k 張圖中的第 X_j 個人的關節點 n ， σ_l 為肢體寬度， $l_{c,k}$ 為肢體長度， P 為影像中的任一點。

4.3 辨識球員背號和球衣顏色

找出投籃的球員後，本研究會將此球員的節點 3(右肩膀)、6(左肩膀)、9(右臀)和 12(左臀)位置框起來，再來辨識該球員背號和球衣顏色(圖 26(b))。

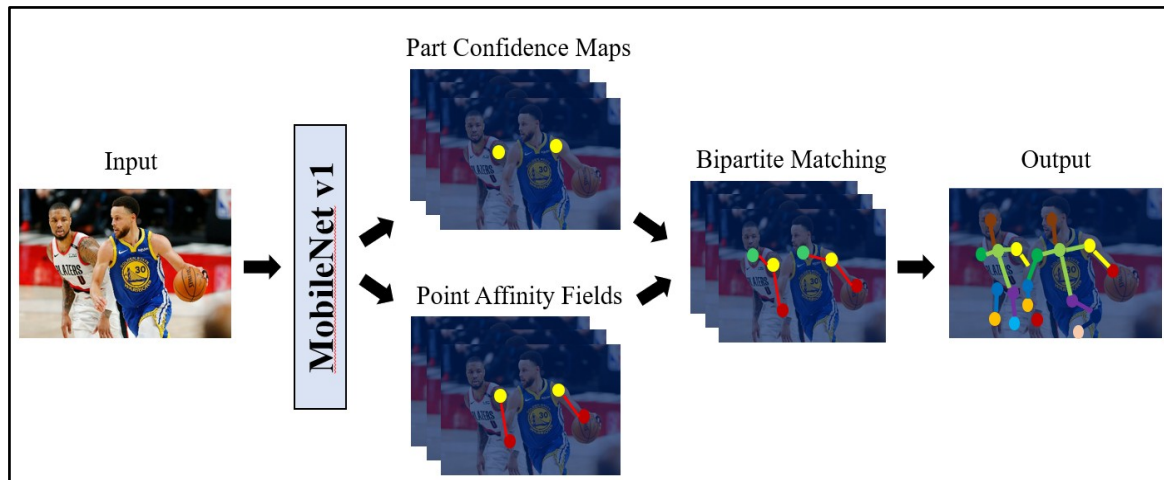


圖 24 訓練 Lightweight OpenPose 過程

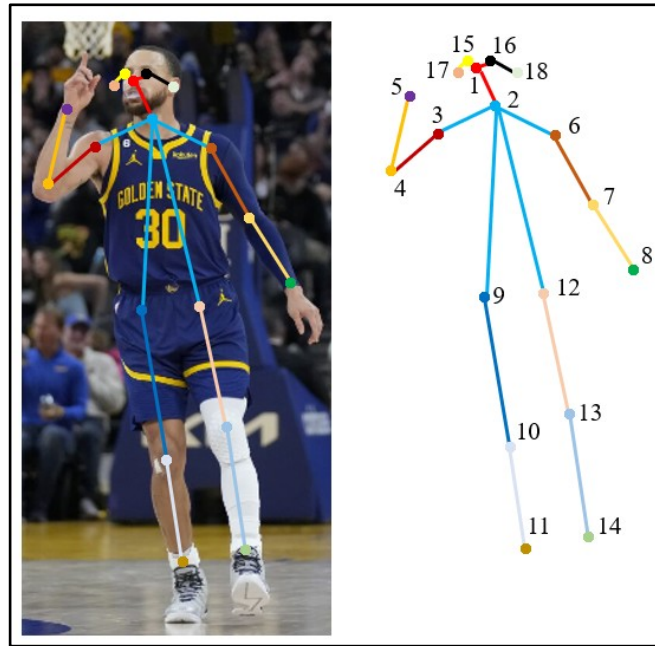
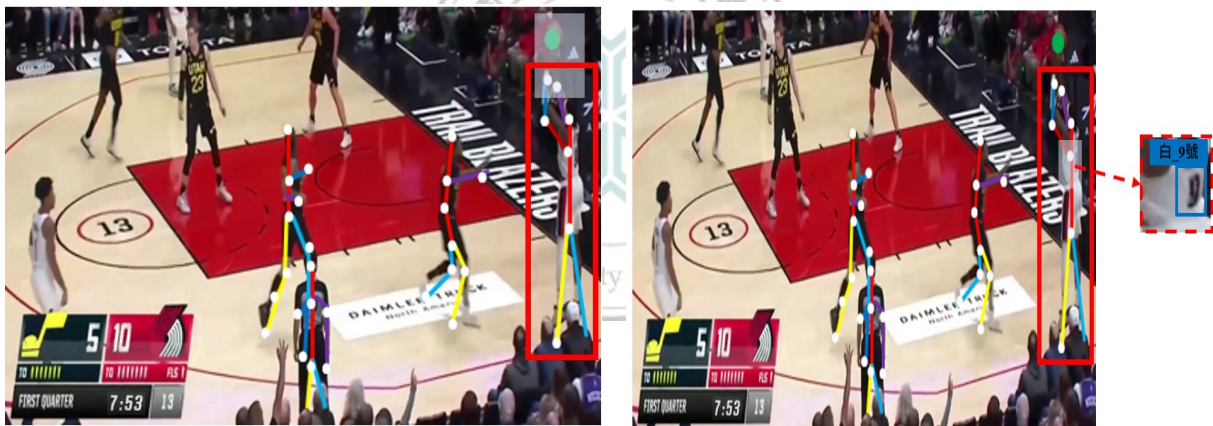


圖 25 人體關節點位置



(a)找出投籃的球員之過程 1(示意圖)

(b)找出投籃的球員之過程 2(示意圖)

圖 26 找出投籃的球員之過程(示意圖)

4.4 得分種類辨識

本章節訓練 3D-CNN 進行得分種類辨識，模型輸入為球員投籃時的畫面，輸出為得分分類，模型架構包含(1)輸入層、(2)卷積層、(3)池化層、(4)全連接層和(5)輸出層，其中訓練 3D-CNN 的過程就如圖 27 所示，首先將 N 張的遠鏡頭比賽畫面輸入到(1)輸入層，假設遠鏡頭的畫面大小為 1280×720 ，且 N 為 10 張，模型則會以 $1280 \times 720 \times 10$ 的輸入維度進入(2)卷積層做訓練，其中 3D-CNN 的卷積核為三維矩陣，用於擷取重要的時空特

徵，並將擷取後的結果輸入至(3)池化層進行降維，以突顯不同影像中的重要特徵區域，提升分類績效，最後透過(4)全連接層整併卷積層和池化層的學習結果，並在(5)輸出層輸出目標類別。而為了讓 3D-CNN 能更好的學習到球員投籃位置以及最終是否投進的資訊，本研究設計了不同的資料前處理方法做實驗，並在實驗模擬章節中找出績效最好的模型，以下會分成四個部分進行說明，(1)降低影像畫面大小、決定間格幀數，(2)訓練單一模型辨識得分種類，(3)訓練混合式模型辨識得分種類，(4)訓練兩個模型辨識得分種類。

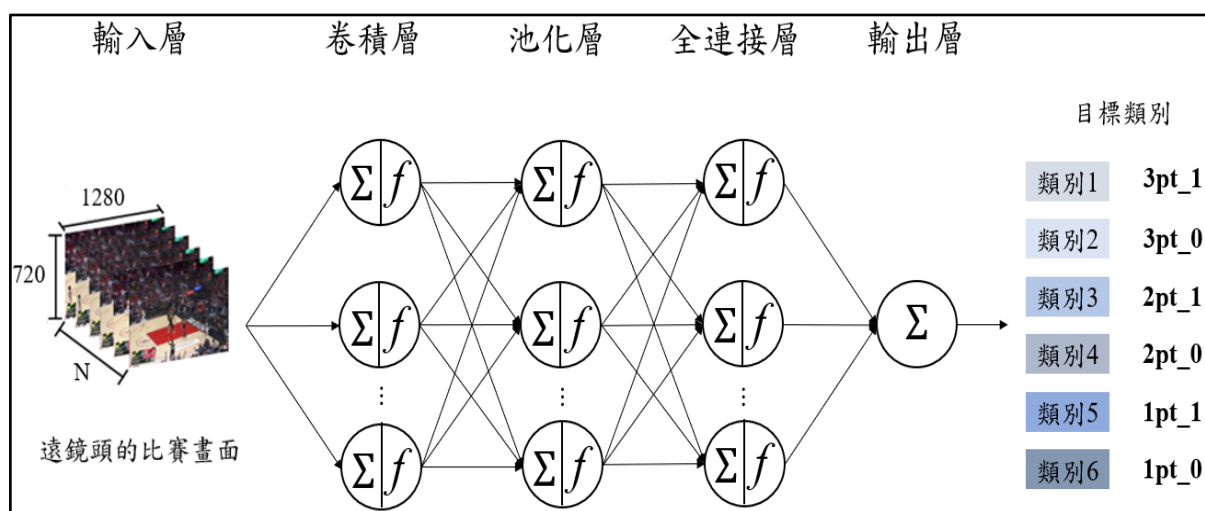


圖 27 訓練 3D-CNN 過程之示意圖

4.4.1 降低影像畫面大小、決定間格幀數

本研究所使用的 3D-CNN 模型是由 C3D 架構進行調整，模型架構如圖 28 所示，此模型是由一層輸入層、四層卷積層、四層最大池化層、一層 Dropout 層、一層 BatchNormalization 層、一層全局池化層、三層全連接層和一層輸出層組成，而為了降低 3D-CNN 的訓練參數量，以及減少較不重要的特徵區域，使模型能聚焦在學習重要特徵，本研究設計了不同的實驗參數來壓縮影像大小、調整間格幀數，以訓練 3D-CNN，目的是要找出分類準確度(Accuracy)最高的影像大小以及最佳的間格幀數，此外，分類績效最佳的影像大小會是後續第 4.4.2~4.4.4 章節實驗的影像尺寸，進一步訓練 3D-CNN，其中設定不同的間格幀數和時間會決定影像數量，本研究是設定 4.5 秒為單位，來決定每次需要輸入幾張影像訓練模型，假設間格幀數為 20 幀，就會有 14 張影像；若間格幀

數為 30 幀時，則會有 9 張影像。



圖 28 3D-CNN 架構

4.4.2 訓練單一模型辨識得分種類

找出最佳的輸入影像大小後，再來是訓練單一模型來辨識得分種類，而為了能再提升模型的分類績效，本研究設計了三種不同的資料前處理方法，分別為(1)改變輸入長度，以找出最適合輸入模型的影像數量，(2)標註球的位置(如圖 29(b)所示)，來增強重要特徵，(3)先做影像二值化，再標註球的位置，目的和資料前處理方法(2)一致，本研究是使用 OpenCV 套件的 TRUNC 和 OTSU 函式來壓縮影像內的數值區間，首先我們會設定一個閾值輸入至 TRUNC 函式，若像素值大於閾值，像素值為閾值，否則像素值不變。圖 30 為壓縮影像前後的資料分布，可以看出壓縮前的資料分布為雙峰分布，數值大多集中在 48 和 210 左右，但在壓縮影像後，數值卻幾乎落在閾值 129，代表壓縮前大多數的數值都超過 129，原因正如圖 29(a)所示，可發現畫面中大部分的範圍為白色的地板，所以才会有圖 30(b)的分布情形。壓縮完影像數值後，再來是標註球的位置(如圖 29(c)所示)，再輸入至模型進行訓練。



(a)原始影像

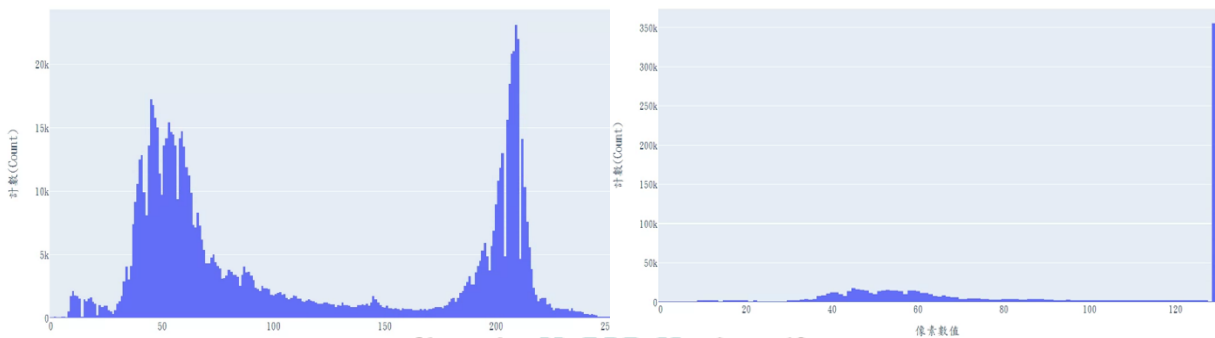


(b)標註球後的影像



(c)二值化+標註球後的影像

圖 29 不同資料前處理下的影像



(a)壓縮影像前的資料分布

(b)壓縮影像後的資料分布

圖 30 壓縮影像前後的資料分布

4.4.3 訓練混合式模型辨識得分種類

本研究將辨識投籃位置和判斷是否得分拆成兩個模型做訓練，以下會說明本章節的建模構思，首先建立 3D-CNN 1 和 3D-CNN 2，這兩個模型架構一致，但輸入的資料集不一樣，3D-CNN 1 的輸入會是前一章節績效最佳的資料前處理後影像，而 3D-CNN 2 的輸入則是籃框下發球線的畫面(如圖 31 所示)，原因是要偵測發球線上是否有球員，在比賽時，當 A 隊球員得分後，B 隊球員就必須到發球線發球，反之，當 A 隊球員沒有得分時，B 隊球員就會直接發動進攻，因此本研究想透過偵測發球線的畫面，輔助模型判斷是否得分。本研究將整張影像中的籃框下發球線畫面框起來並壓縮影像大小，再輸入至 3D-CNN 2 進行訓練，最終將 3D-CNN 1 和 3D-CNN 2 的卷積結果進行合併，攤平後輸出目標類別(如圖 32 所示)。本研究也會使用前一章節的資料前處理方法來訓練模型，分別為(1)改變 3D-CNN 2 輸入長度，(2)僅標註球，不標記球員(如圖 33(b)所示)，(3)先做影像二值化，再標註球或球員的位置(如圖 33(c)、33(d)所示)，再輸入至模型進行訓練。

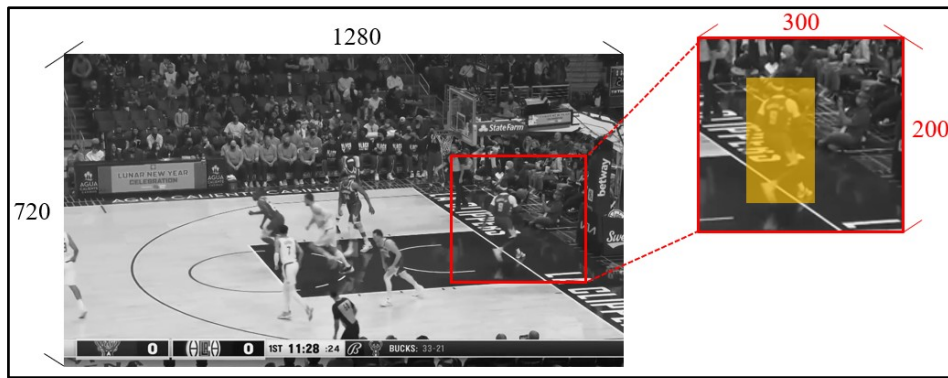


圖 31 籃框下發球線的畫面

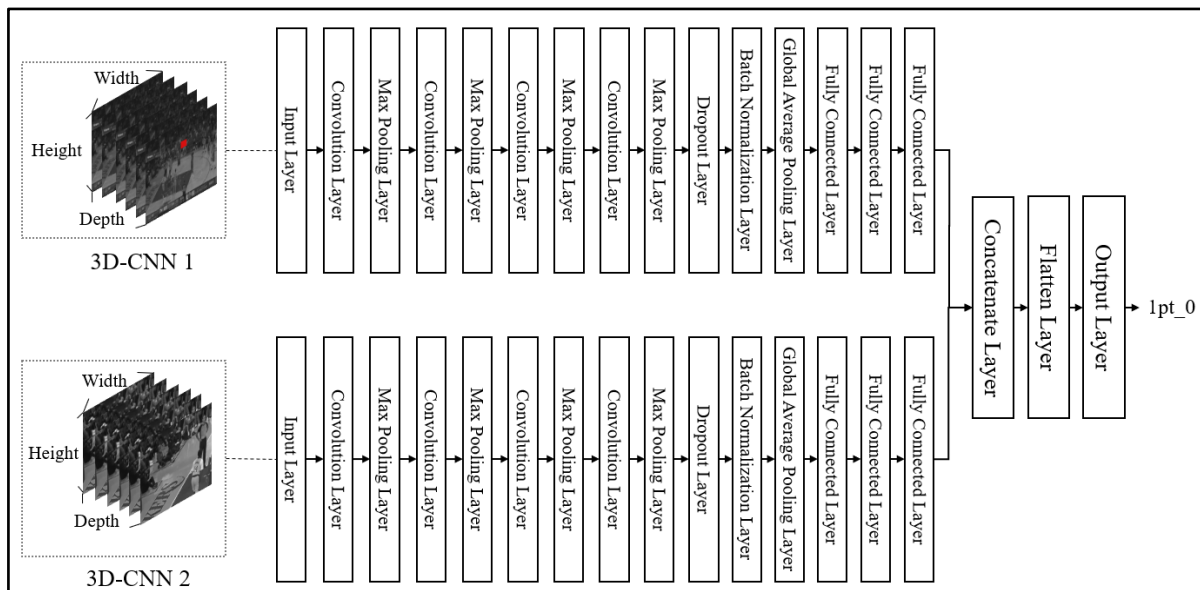
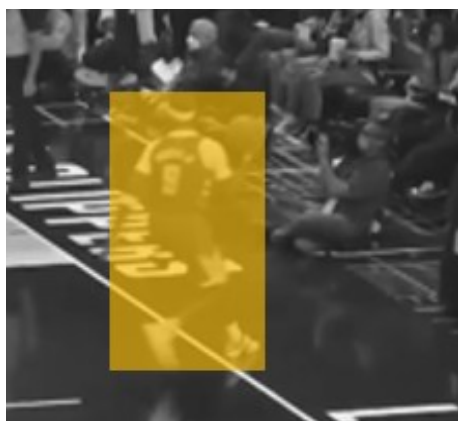


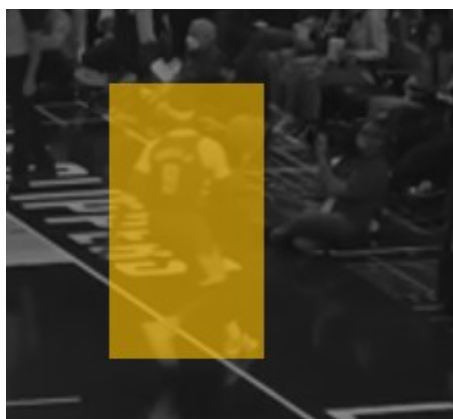
圖 32 混合式 3D-CNN 架構



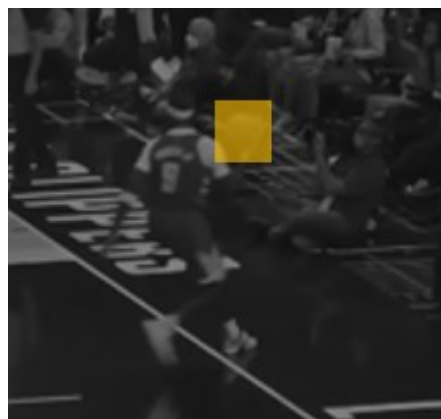
(a)標註完球員位置的畫面



(b)標註完球位置的畫面



(c)二值化+標註球員的畫面



(d)二值化+標註球的畫面

圖 33 不同資料前處理下的影像(3D-CNN 2 輸入)

4.4.4 訓練兩個模型辨識得分種類

本章節也是分成兩個模型來辨識投籃位置和判斷是否得分，只是這兩個模型是獨立的模型，而 3D-CNN 1 的輸出類別是 3pt、2pt 和 1pt，3D-CNN 2 則是得分(1)和未得分(0)，整體框架如圖 35 所示，能看出這兩個模型會分別輸出類別，然後我們再進行合併，舉例來說，假設 3D-CNN 1 的輸出為 2pt，而 3D-CNN 2 的輸出是未得分(0)，那本研究會將此類別轉換成 2pt_0 為最終的預測結果。本研究也使用了前一章節的資料前處理方法來訓練模型，分別為(1)改變輸入長度，但 3D-CNN 1 和 3D-CNN 2 的輸入長度和區間會不一致(2)標註球或是籃框的位置(如圖 34(b)所示)，(3)先做影像二值化，再標註球或籃框的位置(如圖 34(c)所示)，再輸入至模型進行訓練。



(a)原始影像



(b)標註球和籃框後的影像



(c)二值化+標註球和籃框
後的影像

圖 34 不同資料前處理下的影像

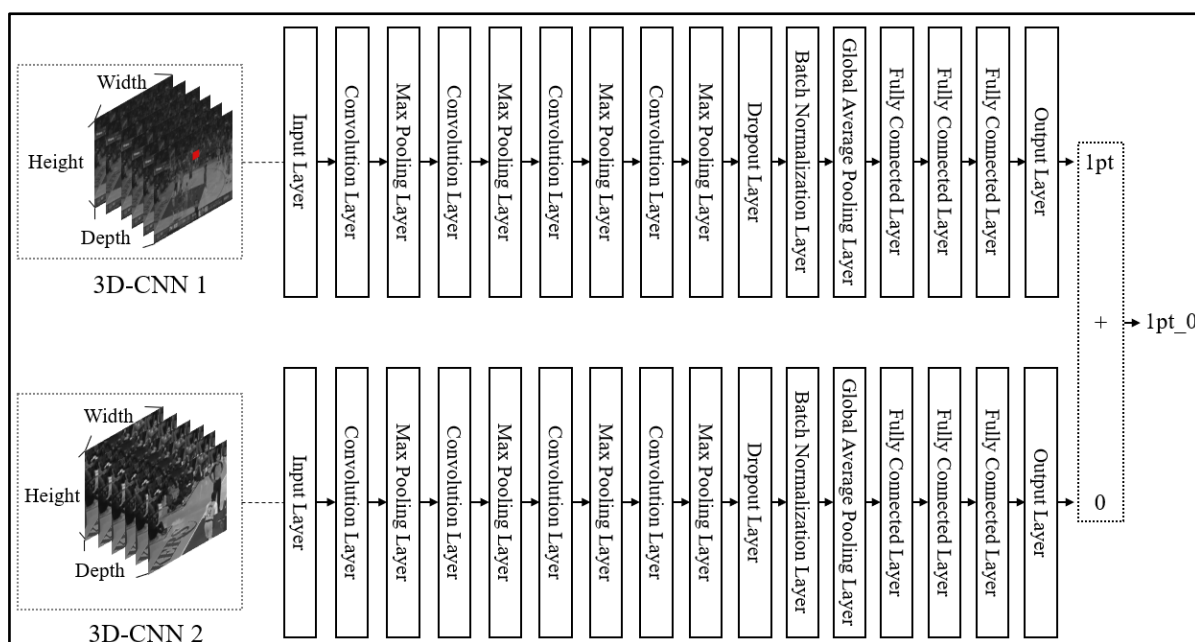


圖 35 兩個 3D-CNN 之框架

經由第 4.4.2~4.4.4 小節的實驗後，本研究會找出最佳的訓練 3D-CNN 方式，即是本研究框架的 3D-CNN 架構來辨識得分種類，最終再將 Lightweight OpenPose 和 3D-CNN 輸出結果進行合併，即可生成文字敘述。

4.5 各模型之評估指標

本研究訓練 CNN 將遠近鏡頭畫面進行分類以及訓練 3D-CNN 進行得分種類辨識，並使用混淆矩陣(表 4)作為評估指標，計算模型預測結果之準確度(Accuracy)、精確度(Precision)、召回率(Recall)以及 F1-score 分數來驗證模型的分類績效，其中準確度(Accuracy)為模型預測正確數量佔整體之比例；精確度(Precision)為預測為遠鏡頭比賽畫面類別(Positive)的資料中，實際也是遠鏡頭比賽畫面類別(Positive)的比例；召回率(Recall)為真實為遠鏡頭比賽畫面類別(Positive)的資料中，預測正確的比例；F1-score 分數為準確度與精確度的調和平均數，其計算方式如公式 4、5、6 和 7。

表 4 混淆矩陣

實際\預測	Positive	Negative
Positive	True Positive (TP)	False Positive (FP)
Negative	False Negative (FN)	True Negative (TN)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$F1-score = \frac{2 \cdot Recall \cdot Precision}{Recall + Precision} \quad (7)$$

本研究訓練 YOLO-v5 來偵測球和籃框的位置，評估指標[29]為 IoU(Intersection over Union)和 mAP(mean Average Precision)；IoU 用來衡量預測邊界框與實際邊界框交集和並集的比值(圖 36)，會介於 0~1 之間，假設將 IoU 閾值設定為 0.5，當 $IoU \geq 0.5$ 時，代表模型有偵測到目標物件，否則為未偵測到目標物件，計算過程如公式 8。AP(Average Precision)是由精確度(Precision)和召回率(Recall)形成的曲線下面積，用來評估各類別的辨識效果，而 mAP 則用來評估所有物件類別的平均 AP(Average Precision)，mAP 越高代表模型預測所有物件類別的效果越佳，計算過程如公式 9。

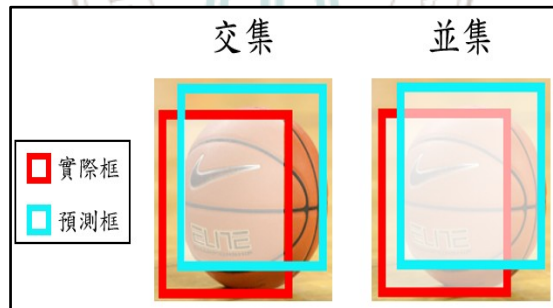


圖 36 預測、實際邊界框之交集和並集

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union} \quad (8)$$

$$mAP = \frac{1}{n} \sum_{k=1}^{k=n} AP_k \quad (9)$$

$AP_k = \text{the Average Precision of class } k$
 $n = \text{the number of classes}$

本研究訓練 Lightweight OpenPose 以找出投籃的球員，該方法使用的評估指標為 OKS(Object Keypoint Similarity)，用於計算預測和實際人體關節點的相似度，若 OKS 為 1，代表模型預測和實際人體關節點的相似度高，OKS 為 0 代表模型預測和實際人體關節點的相似度低，計算過程如公式 10，其中 p 為人的 id、 i 為關節點 id、 d_{pi} 表示實際、預測人體關節點的歐氏距離、 S_p 表示此人在影像中的面積佔比、 σ_i 則表示使用者在標註每個關節點時的標準差，標準差越大代表此關節點越難標註， v_{pi} 為第 p 個人的第 i 個關節點是否有被偵測到、 δ 是將已被偵測到的點位挑選出來進行計算的函數，其中下表 5 為 COCO 資料集平台提供的每個關節點標準差(σ_i)。

$$OKS_p = \frac{\sum_i \exp\left\{-\frac{d_{pi}^2}{2S_p^2\sigma_i^2}\right\} \delta(v_{pi} = 1)}{\sum_i \delta(v_{pi} = 1)} \quad (10)$$

表 5 COCO 資料集平台提供的每個關節點標準差(σ_i)[8]

關節點編號	關節點	標準差(σ_i)	關節點編號	關節點	標準差(σ_i)
1	鼻子	0.026	10	右膝蓋	0.087
2	脖子	0.079	11	右腳踝	0.089
3	右肩膀	0.079	12	左臀	0.107
4	右手肘	0.072	13	左膝蓋	0.087
5	右手腕	0.062	14	左腳踝	0.089
6	左肩膀	0.079	15	右眼睛	0.025
7	左手肘	0.072	16	左眼睛	0.025
8	左手腕	0.062	17	右耳朵	0.035
9	右臀	0.107	18	左耳朵	0.035

第五章 實驗模擬

本研究使用 YouTube 網站[40]上的 NBA 直播影片作為訓練本研究框架的輸入資料集，NBA 文字資料則是輸出資料集，而此框架的架構包含四大部分，第一部分是訓練 CNN 將遠近鏡頭之比賽畫面進行分類，第二部分是訓練 YOLO-v5 來偵測畫面中球和籃框的位置，第三部份是透過 Lightweight OpenPose 找出投籃的球員，第四部份則是使用 3D-CNN 辨識得分種類，再將各模型的預測結果合併成文字敘述，以達到本研究的目標，最終以量化方式來驗證各模型績效、運算時間以及合併成文字後的效果，以下會分成八個部分進行介紹，(1)資料集介紹，(2)決定間格幀數大小，(3)CNN 在將遠近鏡頭之比賽畫面進行分類的效果，(4)YOLO-v5 在偵測畫面中球和籃框位置的效果，(5)Lightweight OpenPose 辨識場上球員的效果，(6)不同資料前處理方法對 3D-CNN 的預測效果，(7)將各模型預測結果合併成文字敘述的效果，(8)模擬平行運算所需的電腦數量。本研究的所有實驗皆是以 Python 程式語言進行實作，並且操作在 Intel(R) Core(TM) i9-9940X CPU @ 3.30GHz 的 CPU、兩顆 Nvidia TITAN RTX 24GB 的 GPU、搭配約 128GB 記憶體以及 404GB 的容量，並使用 Windows 10 的作業軟體來完成。

5.1 資料集介紹

本研究所使用的資料集來源於 YouTube 網站[40]，總共收集了 42 場 NBA 直播影片，而目前僅使用 10 場比賽進行實驗模擬，詳細資訊如表 6 所示，其中有 9 場比賽用於訓練本研究提出的框架，1 場比賽則用來測試此框架的預測效果。

表 6 實驗模擬所用到的直播影片

訓練\測試	日期	對戰隊伍	影片長度	幀數(fps)
訓練	2010-05-27	洛杉磯湖人 vs 鳳凰城太陽	1:46:44	30
訓練	2018-06-06	克里夫蘭騎士 vs 金洲勇士	1:46:47	30
訓練	2020-10-11	邁阿密熱火 vs 洛杉磯湖人	2:14:09	30
訓練	2021-10-23	密爾瓦基公鹿 vs 聖安東尼奧馬刺	1:38:45	60
訓練	2022-02-04	達拉斯獨行俠 vs 費城 76 人	1:47:55	60
訓練	2022-02-05	沙加緬度國王 vs 奧克拉荷馬雷霆	1:34:55	60
訓練	2022-02-08	休士頓火箭 vs 紐澳良鵜鶘	1:55:08	60
訓練	2022-03-05	洛杉磯湖人 vs 金洲勇士	1:53:35	60
訓練	2022-03-06	猶他爵士 vs 奧克拉荷馬雷霆	1:42:14	60
訓練	2022-02-06	密爾瓦基公鹿 vs 洛杉磯快艇	1:41:05	60

5.2 決定間格幀數大小

為了降低模型訓練量以及減少電腦的處理時間，本研究設計多個間格幀數來擷取影像，分別為 6、10、12、15、20、30 和 60 幀，並透過實驗模擬找出較合適的間格幀數，以達到兩個目的，(1)維持本研究框架的預測效果，(2)降低電腦的處理時間。而本章節著重在檢視電腦的處理時間，找出較合適的間格幀數後，在後續章節探討本研究框架的預測效果。本研究使用密爾瓦基公鹿隊和洛杉磯快艇隊在 2022 年 2 月 6 號的直播影片進行實驗，影片長度為 1 小時 41 分 05 秒，影片幀數為 60 幀。實驗結果如下表 7 所示，可以發現只有在間格幀數為 20 幀以上時，電腦的處理時間和影片時間差異較小，但當間格幀數為 60 幀時，雖然能大幅降低電腦的處理時間，但是會遺失大量的資訊，以下透過本研究觀察到的其中一個進攻回合進行說明，圖 37 為公鹿隊在第一節剩下 6 分 24 秒時的進攻回合，其中 Frame 1 為公鹿隊大前鋒 Khris Middleton 從三分線外出手的畫面，Frame 2 則是球在接近籃框時的過程，但到了 Frame 3 的畫面時，球就已經在地上了，從這次的進攻回合中，可以得知投籃的球員是誰，但無法判斷球是否被投進，而前述現

象可能會使本研究框架無法學習到關鍵幀的資訊，因此本研究的後續實驗僅會考慮間格幀數為 20 和 30 幀時的模型績效。

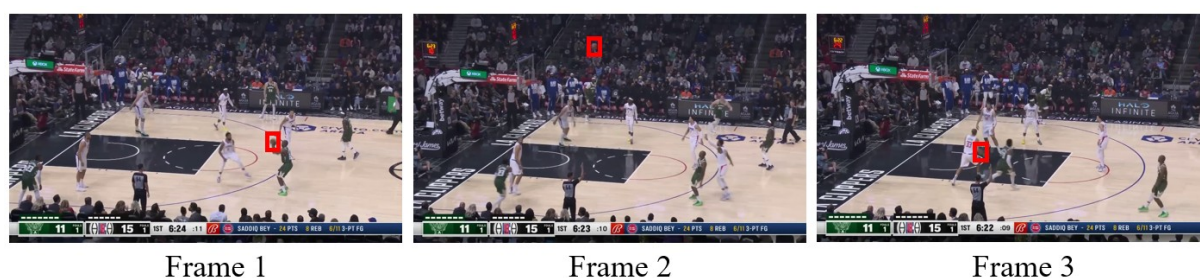


圖 37 間格幀數為 60 幀時的畫面

表 7 電腦在不同間格幀數下的處理時間

間格幀數	電腦處理時間(小時)	評估結果	評估結果為不佳的原因
6	11.7	不佳	電腦處理時間太久
10	7.08	不佳	電腦處理時間太久
12	4.3	不佳	電腦處理時間太久
15	3.56	不佳	電腦處理時間太久
20	2.01	尚可接受	
30	1.44	尚可接受	
60	0.47	不佳	資訊量過少

5.3 CNN 在將遠近鏡頭之比賽畫面進行分類的效果

再來是透過 CNN 將遠近鏡頭的比賽畫面進行分類，首先本研究會從表 6 的訓練集影片中，隨機抽取 7 部影片做訓練，2 部影片用來測試模型，準備好預訓練和測試的影片後，接著本研究會將資料整理到訓練 CNN 的過程，分成三大部分進行說明，(1)將角點偵測擷取到的影像進行分類，類別 0 代表近鏡頭比賽畫面(圖 38(b))，類別 1 則代表遠鏡頭比賽畫面(圖 38(a))，本研究最終隨機挑選 9000 張影像來訓練、測試模型，表 8 為各類別在訓練、測試集的影像數量，(2)建立 CNN 模型架構以及設定訓練參數，本研究建立的模型架構是根據經典的 CNN 模型 Alexnet 進行調整，最終的模型架構如圖 39 所示，此模型是由一層輸入層、三層卷積層、三層最大池化層、一層攤平層、一層 Dropout

層、兩層全連接層和一層輸出層組成，並且使用 Adam 優化器、Epoch 設定為 100、Batch size 為 32 來訓練模型，(3)在訓練模型之前，本研究將所有影像轉換成灰階影像，以降低訓練模型的負荷量，然後再統一除以 255，達到正規化的效果，最終再輸入至模型進行訓練。模型的測試績效如表 9 所示，可以看出大多數的樣本都被分類在正確的類別上，模型預測的準確度(Accuracy)為 96.44%、精確度(Precision)為 96.82%、召回率(Recall)為 96.14%以及 F1-score 分數為 96.48%，而本研究認為 CNN 分類佳的原因是，遠鏡頭和近鏡頭比賽畫面中的球員特徵是截然不同的，而這兩種比賽畫面的差異可分成兩個，一、球員佔整張影像的比例，二、影像中的球員數量，因此 CNN 僅需要學習到上述兩點就能正確判斷各影像的類別。訓練完 CNN 後，接著會將遠鏡頭比賽畫面輸入至 YOLOv5、Lightweight OpenPose 和 3D-CNN 進行訓練。

表 8 各類別在訓練、測試集的影像數量

資料集\類別	類別 0	類別 1
訓練	2291	2209
測試	2218	2282

表 9 CNN 的測試績效

總樣本數(4500 張)		預測值	
		近鏡頭比賽畫面	遠鏡頭比賽畫面
實際值	近鏡頭比賽畫面	2146	88
	遠鏡頭比賽畫面	72	2194



(a)遠鏡頭的比賽畫面



(b)近鏡頭的比賽畫面

圖 38 遠近鏡頭的比賽畫面

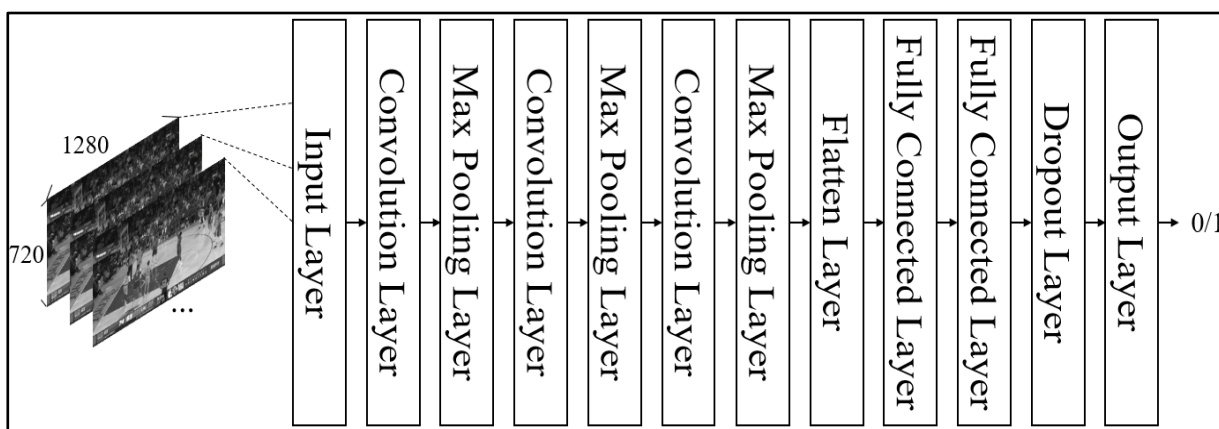


圖 39 CNN 架構

5.4 YOLO-v5 在偵測畫面中球和籃框位置的效果

本研究訓練 YOLOv5 找出球與籃框位置最近的畫面，然而在資料前處理方面，本研究使用前一章節分類出的遠鏡頭比賽畫面，透過 MakeSense.AI 網站標記球和籃框的位置，總共標記了 4013 張影像，包含球員在運球、投籃、傳球等動作的畫面，並且將資料集分成訓練、測試和驗證集三個部分，其中訓練集有 2810 張、測試集有 602 張、驗證集則有 601 張影像，標記完影像後的輸出資料就會包含物件的類別和位置資訊。而本研究測試 YOLOv5 各模型的辨識效果，原因是想找出辨識效果和偵測時間較符合本研究需求的模型，由模型參數量大小排序下來，分別為 YOLOv5n、YOLOv5s、YOLOv5m、YOLOv5l 和 YOLOv5x，至於模型的參數包含(1)輸入至模型的影像大小為 640×640 ，(2)設定 Epoch 和 Batch size 為 50 來訓練模型，最終在測試模型時，會以 0.5 到 1.0 的 IoU

閾值來檢視模型的辨識效果，並且將信心分數設定為最低值 0.25，而表 10、11 為 YOLOv5 各模型的驗證、測試集績效以及平均偵測一張影像所花費的時間，可從此表中得知三件事情，(1)所有模型在辨識球的 AP(Average Precision)值都會比籃框差，原因是在比賽的過程中，球時常會被場上球員擋住或是正在移動的過程，使得模型難以辨識球的位置(如圖 40、41 所示)，這也是本研究將信心分數設定為最低值 0.25 的原因，但同時又遇到一個問題是，模型可能會偵測多顆球在一張影像裡面(如圖 42 所示)，針對此問題本研究會選擇信心分數最高的視為球。相較之下，籃框的位置在影像畫面中較為固定且不會被場上球員擋住，讓模型在辨識籃框上是較為容易的，(2)所有的模型平均都需要 2 至 3 秒的時間來偵測一張影像，並沒有太大的差異，(3)YOLOv5m 在所有模型中的辨識效果是最佳的，測試集的 mAP 高達 94.23%，因此本研究使用 YOLOv5m 作為目標模型，至於考慮到本研究的目標是要偵測球員投籃到球與籃框最近的畫面，於是本研究延續使用第 5.2 章節的測試影片，首先找出所有投籃過程的畫面，再輸入至 YOLOv5m 偵測球與籃框的位置，辨識效果如表 12 所示，可以發現球的 AP 值從原先的 91.01%下降到 79.52%，使得 mAP 值從 94.23%降到 89.74%。最終本研究將所有球與籃框最近的距離紀錄下來(如圖 43 所示)，並統整成三個條件，以便於找出預分析的影像，(1)球與籃框的距離必須小於等於 200，(2)找到球與籃框位置最近的畫面時，球的 Y 值必須大於籃框 Y 值(如圖 44 所示)，(3)若在連續 7 幀內，出現兩次以上球與籃框距離小於 200 的情形，本研究會選擇距離最小的畫面做分析。

表 10 YOLO 驗證集績效

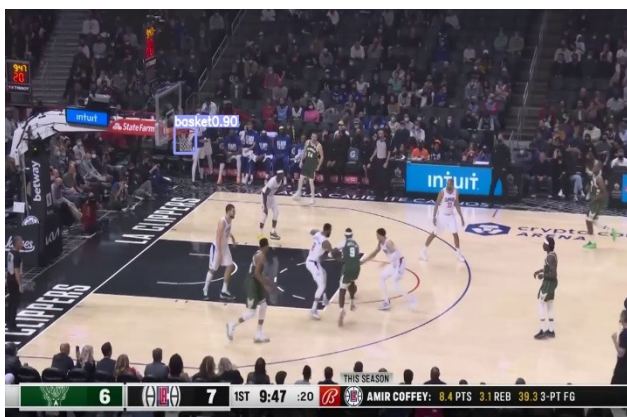
績效\模型	YOLOv5n	YOLOv5s	YOLOv5m	YOLOv5l	YOLOv5x
球(AP)	85.86%	88.17%	90.9%	91.3%	89.86%
籃框(AP)	95.55%	95.55%	97.48%	96.5%	96.32%
mAP	90.7%	91.9%	94.19%	94%	93.1%
總辨識時間(秒)	1712.85	1718.86	1833.05	1869.11	1923.2
平均辨識時間(秒)	2.85	2.86	3.05	3.11	3.2

表 11 YOLO 測試集績效

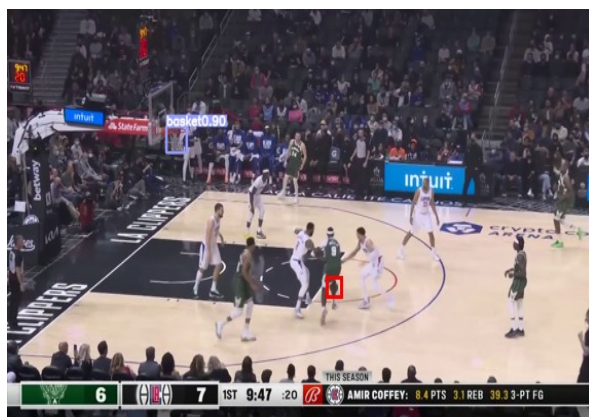
績效\模型	YOLOv5n	YOLOv5s	YOLOv5m	YOLOv5l	YOLOv5x
球(AP)	84.13%	87.12%	91.01%	89.92%	89.72%
籃框(AP)	95.13%	95.88%	97.38%	97%	96.07%
mAP	89.63%	91.5%	94.23%	93.46%	92.9%
總辨識時間(秒)	1703.66	1715.7	1818.04	1890.28	1938.44
平均辨識時間(秒)	2.83	2.85	3.02	3.14	3.22

表 12 YOLOv5m 偵測球員投籃的過程

績效\模型	YOLOv5m
球(AP)	79.52%
籃框(AP)	99.96%
mAP	89.74%



(a)YOLOv5m 偵測此影像的結果

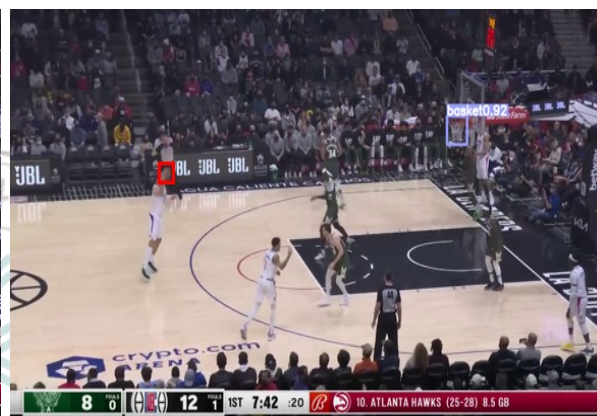


(b)紅框為球的實際位置

圖 40 球被場上球員擋住的畫面



(a)YOLOv5m 偵測此影像的結果



(b)紅框為球的實際位置

圖 41 球正在移動的畫面

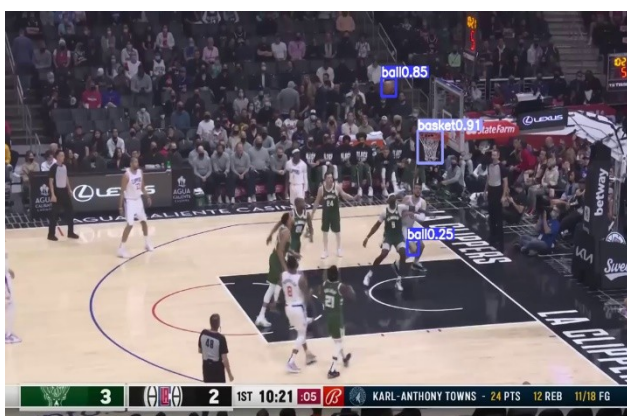


圖 42 YOLOv5m 偵測多顆球的畫面

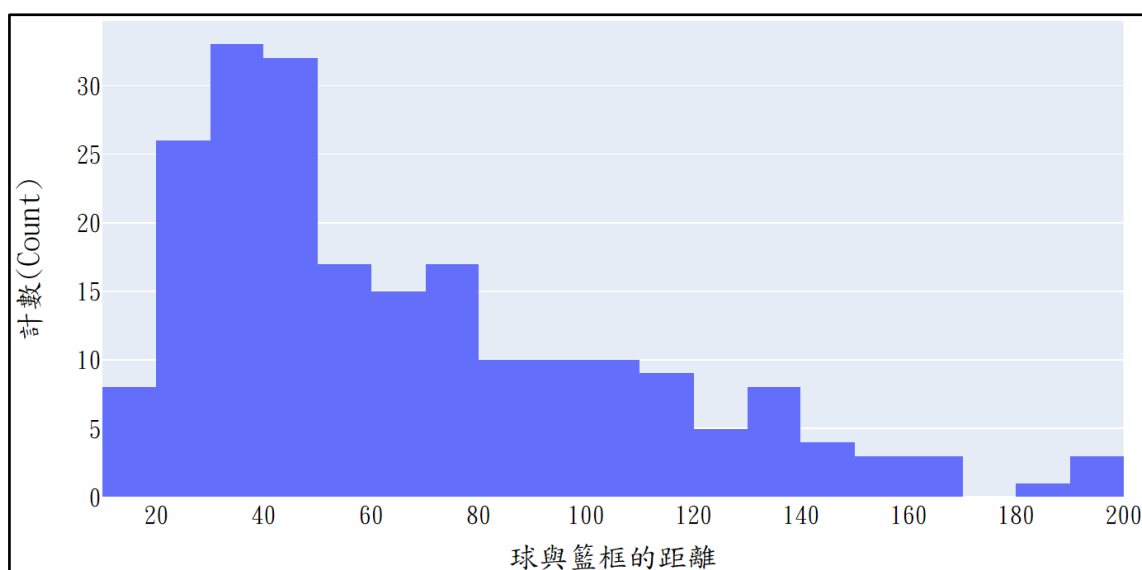


圖 43 球與籃框距離之視覺化

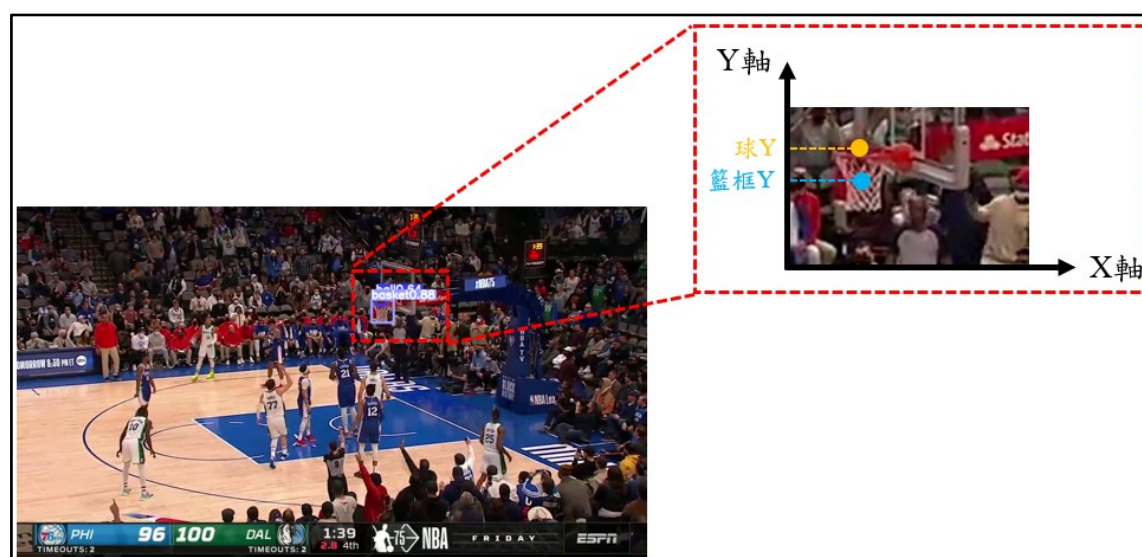


圖 44 球與籃框位置最近的畫面

5.5 Lightweight OpenPose 辨識場上球員的效果

本章節會訓練 Lightweight OpenPose 來辨識場上的球員，並且找出投籃的球員(圖 45)。首先本研究延續使用前一章節辨識出的球與籃框位置最近的畫面，並透過 COCO Annotator 標記場上球員的關節點位置，標記順序如下：鼻子、左眼睛、右眼睛、左耳朵、右耳朵、左肩膀、右肩膀、左手肘、右手肘、左手腕、右手腕、左臀、右臀、左膝蓋、右膝蓋、左腳踝和右腳踝，共 17 個關節點，然而在標記資料時，時常會有無法判斷關節點位置的情形，所以此平台提供關節點可視性(Visibility)的選項，假設今天要標

註 A 球員的左手腕位置，當我們無法在畫面中找到左手腕的位置，會設定 Visibility 為 0；如果是在不確定的情況下推估左手腕位置，則會設定 Visibility 為 1；若是能正確判斷左手腕在畫面中的位置，則設定 Visibility 為 2，而本研究共標記 750 張影像，並將資料集切割成訓練、測試和驗證集，其中訓練集有 500 張、測試集有 100 張、驗證集則有 150 張影像，至於模型架構是使用 Dilated MobileNet v1 來學習人體關節點以及肢體連接的情形，本研究將 Batch size 設定為 40、Epoch 設定為 280、輸入的關節點熱力圖數量為 19，PAF 數量為 38 來訓練模型，評估指標則是使用 OKS(Object Keypoint Similarity)和 mAP，OKS 用於計算預測和實際關節點的相似度，本研究會設定 0.5 到 1.0 的 OKS 閾值，並使用 mAP 來檢視模型辨識場上球員的效果。最終測試完模型的 mAP 為 90%，表示模型有 90%的準確率能辨識影像中的場上球員，而本研究也延續使用前章節找出的球與籃框位置最近的畫面，將此畫面再向前推 1 至 6 幀，並紀錄每張畫面中球與場上球員的節點 5(右手腕)、8(左手腕)距離(如圖 46 所示)，經由統計數據後，本研究統整成兩個條件，幫助我們找出投籃的球員，(1)球與場上球員的節點 5 或 8 的距離必須小於等於 80，(2)若有多個球員的節點 5 或 8 的距離符合條件 1，本研究會選擇距離最小的視為投籃的球員。找出投籃的球員後，接著要辨識該球員的背號和球衣顏色，首先本研究會將影像轉換成灰階影像，再把節點 3(右肩膀)、6(左肩膀)、9(右臀)和 12(左臀)位置框起來進行辨識，但在辨識的過程中，發現即使已經得知投籃球員的位置，有時也未必能精準的辨識他的背號，原因主要分成兩點：(1)投籃球員的背號剛好沒有面向鏡頭(圖 47)，(2)被其他球員擋住了(圖 48)，為了解決上述兩個問題，本研究將球員投籃時的畫面，再向前、後推 5 幀來辨識此球員的背號(圖 49)，若模型在向前、後推幀的過程中，就已經辨識出球員背號，那推幀過程就會被停止，希望藉由提供投籃球員在相鄰幀的位置，來改善模型在辨識背號上的問題，至於在辨識此球員的球衣顏色時，本研究的作法是去辨識球衣的深淺色，而不是辨識紅色、藍色等色彩顏色，原因是雙方球隊在比賽時，穿的球衣顏色會有明顯的對比，讓球迷們和體育台主播能清楚判斷場上分別是哪些球員，所以僅需要判斷目前投籃球員是穿深色還是淺色球衣，即可得知此球員的所屬球隊，而本研究目前是設定 127 為閾值，若球衣的像素值大於 127，代表是淺色球衣，否則為深色球衣。



圖 45 透過 Lightweight OpenPose 找出投籃球員的示意圖

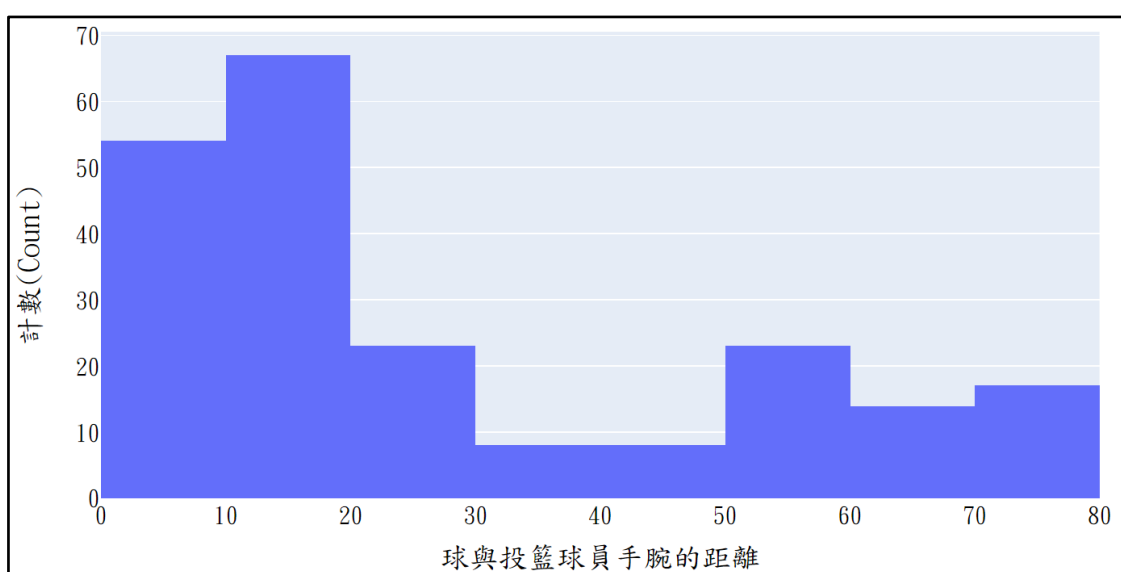


圖 46 球與投籃球員手腕距離之視覺化

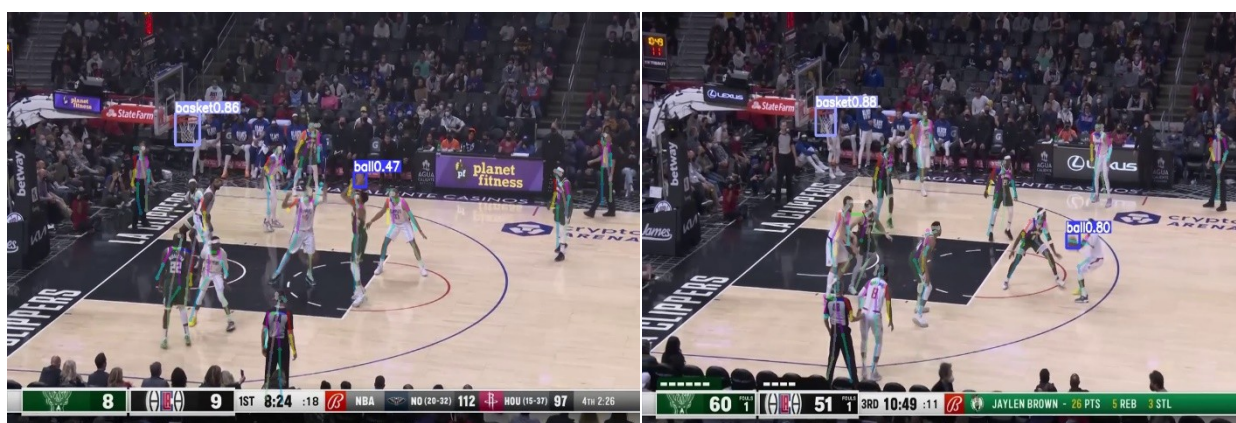


圖 47 投籃球員背號未面向鏡頭的畫面

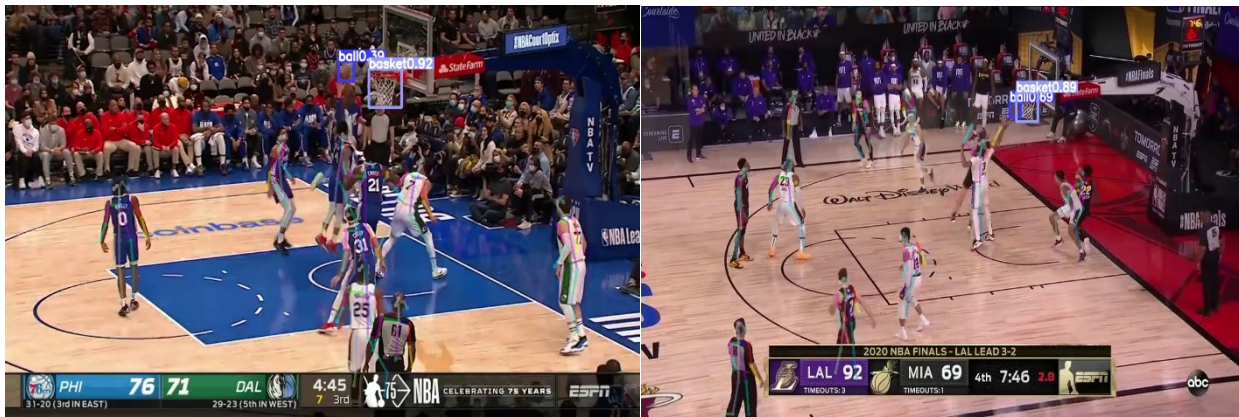


圖 48 投籃球員被其它球員擋住的畫面

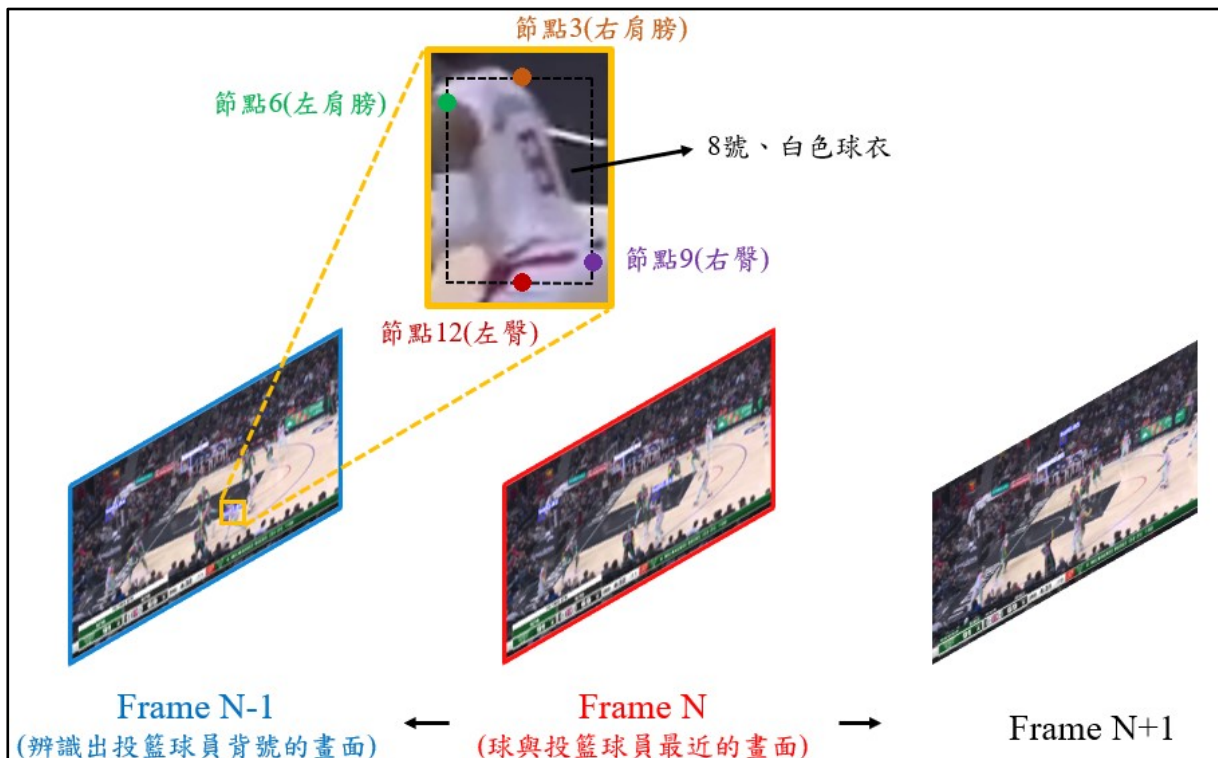


圖 49 辨識投籃球員背號的過程

5.6 不同資料前處理方法對 3D-CNN 的預測效果

經由第 5.4 和 5.5 章節找出球員投籃到球與籃框最近的畫面後，本章節將訓練 3D-CNN 來辨識得分種類，其中輸入模型的資料維度為(N,Depth,Width,Height,Channel)，第一個維度 N 代表訓練樣本的數量、第二個維度 Depth 代表每次輸入幾張影像訓練模型，本研究預設為 14 張，原因是在觀察資料時，發現投籃球員從出手到交換球權的過程，最長時間會在 4.5 秒左右，並且在間格幀數為 20 的情況下，每秒就會擷取 3 張影像，因此 4.5 秒乘以 3 約為 14 張，第二個維度 Depth 才會設定為 14，再來會透過實驗模擬找

出最佳參數，第三、四個維度 Width 和 Height 分別代表影像的寬和高，最後一個維度是通道(Channel)。模型的輸出資料共有 6 個類別，分別為 3pt_0、3pt_1、2pt_0、2pt_1、1pt_0 和 1pt_1，而各類別數量如表 13 所示，可以發現類別 2pt_1 的數量是最多的，類別 1pt_0 的數量則是最少的，本研究將資料集切割成訓練和測試集，比例為 8:2，訓練集有 712 組、測試集則有 179 組。為了讓 3D-CNN 能更好的學習到球員投籃位置以及最終是否投進的資訊，本研究設計了不同的資料前處理方法做實驗，並找出績效最好的模型，以下會分成四個部分進行說明，(1)降低影像畫面大小、決定間格幀數，(2)訓練單一模型辨識得分種類，(3)訓練混合式模型辨識得分種類，(4)訓練兩個模型辨識得分種類。

表 13 訓練 3D-CNN 的輸出類別數量

輸出類別	1pt_0	1pt_1	2pt_0	2pt_1	3pt_0	3pt_1
影像數量	1148	1400	2645	2996	2576	1666
組數	82	100	191	214	184	119

5.6.1 降低影像畫面大小、決定間格幀數

本研究設計了 7 個實驗參數來訓練 3D-CNN 模型，分別為 20×20、40×40、60×60、80×80、100×100、160×160 和 240×240 來壓縮影像大小，並找出分類準確度(Accuracy)最高的影像大小。間格幀數參數則設定為 20 和 30，並且使用 Adam 優化器、Epoch 設定為 100、Batch size 為 20 來訓練模型，最終測試結果如表 14、15 所示，可以發現間格幀數為 20 幀且影像大小為 80×80 時的分類準確度(Accuracy)最高，達到 77.39%，其中本研究認為模型在間格幀數 30 幀的分類效果普遍比 20 幀低的原因是，前者提供整個投籃事件的資訊比後者來得少，使模型較難完整的學習到投籃事件過程，所以分類效果較差是相當合理的。在後續的實驗都會將影像大小從 1280×720 壓縮至 80×80，並且使用 20 幀的間格幀數來訓練模型。

表 14 間格幀數為 20 幀、不同影像大小的 3D-CNN 績效

影像大小	Accuracy	Recall	Precision	F1 score
20×20	32.96%	32.96%	42.16%	35.09%
40×40	36.31%	36.31%	73.5%	43.13%
60×60	51.11%	51.11%	55.02%	51.85%
80×80	77.39%	77.39%	77.48%	77.25%
100×100	70.05%	70.05%	71.13%	70.08%
160×160	63.42%	63.42%	67.6%	63.65%
240×240	54.37%	54.36%	55.13%	54.54%

表 15 間格幀數為 30 幀、不同影像大小的 3D-CNN 績效

影像大小	Accuracy	Recall	Precision	F1 score
20×20	33.46%	33.46%	34.07%	33.65%
40×40	34.08%	34.08%	34.91%	34.79%
60×60	55.26%	55.26%	56.13%	55.84%
80×80	74.44%	74.43%	78.66%	75.06%
100×100	63.48%	63.48%	63.54%	63.51%
160×160	49.7%	49.7%	55.95%	50.21%
240×240	47.93%	47.93%	61.86%	51.08%

5.6.2 訓練單一模型辨識得分種類

本章節會設計三種不同的資料前處理方法訓練單一模型來辨識得分種類，分別為(1)改變輸入長度，實驗參數分別為第 1~14、1~13、1~12、1~11 和 1~10 幀，(2)標註球的位置，(3)先做影像二值化，再標註球的位置，總共有 15 個實驗。最終本研究將這 15 個實驗結果紀錄下來，並統整資料前處理方法(1)(2)(3)的最佳結果成表 16，從表 16 中可以得知三件事情，一、每組實驗最佳的輸入長度皆為第 1~14 幀，二、標註球位置的測試績效會比沒有標註球位置佳，三、資料前處理方法(3)對於訓練模型的分類效果最佳，測試的分類準確度(Accuracy)達到 79.28%，其中從表 17 可以看出兩件事，(1)相較於辨識投籃位置，模型在預測是否得分的效果較差，(2)類別 3pt_0 在所有類別中，被分錯的次數是最高。

表 16 單一模型辨識得分種類的測試績效

輸入長度	影像二值化	標註球位置	Accuracy	Recall	Precision	F1_score
1~14			77.39%	77.39%	77.48%	77.44%
1~14		✓	78.11%	78.11%	79.28%	78.44%
1~14	✓	✓	79.28%	79.28%	80.32%	79.56%

表 17 測試 3D-CNN 之混淆矩陣

總樣本數(179 組)		預測值					
		3pt_0	3pt_1	2pt_0	2pt_1	1pt_0	1pt_1
實際值	3pt_0	31	2	1	2	0	3
	3pt_1	2	22	1	1	0	1
	2pt_0	3	0	31	2	0	1
	2pt_1	3	3	1	35	1	0
	1pt_0	0	1	0	0	5	7
	1pt_1	0	0	0	0	0	20

5.6.3 訓練混合式模型辨識得分種類

針對前一章節末所提出的兩個問題，本研究認為要讓 3D-CNN 同時辨識投籃位置和判斷是否得分，是相當具有挑戰性的，因此拆成兩個 3D-CNN 模型進行訓練，模型架構如圖 32 所示，與前一章節的不同點在於，本章節是個別訓練 3D-CNN 1 和 3D-CNN 2，並且輸入的影像長度和區間都不一致，最終將這兩個模型的卷積結果進行合併，攤平後輸出目標類別。資料前處理方法共有三種，分別為(1)改變 3D-CNN 2 輸入長度，實驗參數分別為第 9~14、9~13、9~12、9~11、10~14、10~13、10~12、11~14 和 11~13 幀，(2)僅標註球，不標記球員，(3)先做影像二值化，再標註球或球員的位置。統整上述實驗，總共有 36 個實驗要來測試混合式 3D-CNN 績效，最終也統整資料前處理方法(1)(2)(3)的最佳結果成表 18，從表 18 中可以得知當輸入至模型的長度為第 9~14 幀、標註球員位置且不需要做影像二值化時的分類績效最佳，但分類準確度(Accuracy)卻從原先的 79.28%下降到 66.48%，表示模型仍然沒有學起來，看完表 19 的分類結果後，本研究推測模型學不好的原因是，並非只有在投籃球員得分後，才会有球員出去發球線發球，以下有兩種情形可以做說明，(1)球員在上籃或灌籃時不小心衝出發球線外(圖 50)，球沒進但模型預測進球，(2)球員投籃後沒進，而球也出界了(圖 51)，所以球權回到另一隊手上，必須重新發球以開始此回合的進攻，由於模型偵測發球線上有球員出現，所以會預測進球。由上述兩點可得知，輸入籃框下發球線的畫面來訓練 3D-CNN 2，在部份情況下會干擾模型做決策，反而會影響整體的分類績效。

表 18 混合式模型辨識得分種類的測試績效

輸入長度	影像二值化	標註	Accuracy	Recall	Precision	F1_score
9~14		球員	66.48%	66.48%	71.92%	67.23%
9~13		球	56.67%	56.67%	64.3%	58.85%
10~14	✓	球員	61.11%	61.11%	71.36%	64.82%
10~12	✓	球	60%	60%	68.96%	61.62%

表 19 測試混合式 3D-CNN 之混淆矩陣

總樣本數(179 組)		預測值					
		3pt_0	3pt_1	2pt_0	2pt_1	1pt_0	1pt_1
實際值	3pt_0	36	4	3	0	0	0
	3pt_1	2	19	0	2	0	0
	2pt_0	8	3	23	11	0	0
	2pt_1	2	5	4	19	0	2
	1pt_0	0	0	0	0	8	12
	1pt_1	0	0	1	0	1	14

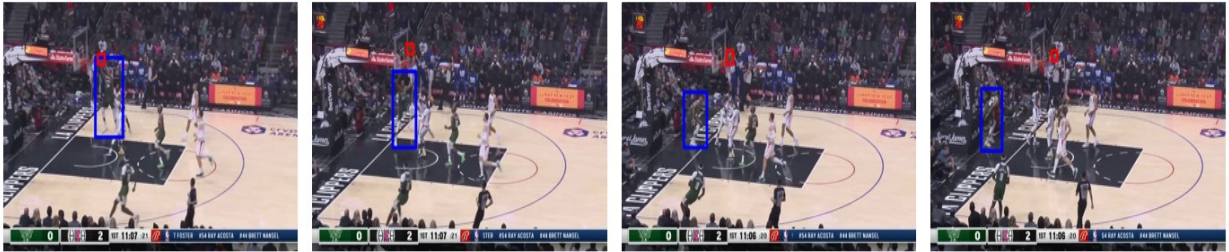


圖 50 球員在上籃或灌籃時不小心衝出發球線外的畫面

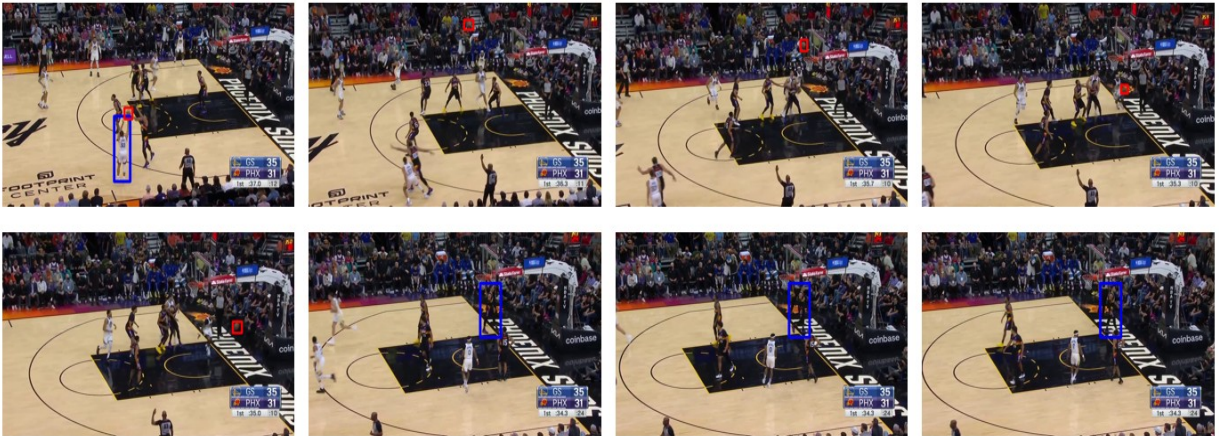


圖 51 球員投籃後沒進，然後球出界的畫面

5.6.4 訓練兩個模型辨識得分種類

到目前為止，最佳的模型分類績效只有 79.28%，為了再提高分類準確度(Accuracy)，本研究也是分成兩個模型來辨識投籃位置和判斷是否得分，只是這兩個模型是獨立的模型，而 3D-CNN 1 的輸出類別是 3pt、2pt 和 1pt，3D-CNN 2 則是得分(1)和未得分(0)，整體框架如圖 35 所示，能看出這兩個模型會分別輸出類別，然後我們再進行合併，舉例來說，假設 3D-CNN 1 的輸出為 2pt，而 3D-CNN 2 的輸出是未得分(0)，那本研究會將此類別轉換成 2pt_0 為最終的預測結果。本研究也使用了前一章節的資料前處理方法來訓練模型，分別為(1)改變輸入長度，3D-CNN 1 的實驗參數為第 1~3、1~4、1~5、1~6、1~7 和 1~8 幀，而 3D-CNN 2 的實驗參數則是第 3~12、3~11、3~10、3~9、3~8 和 3~7 幀(2)標註球或是籃框的位置，(3)先做影像二值化，再標註球或籃框的位置。統整上述實驗，3D-CNN 1 和 3D-CNN 2 都各有 24 個實驗，而統整資料前處理方法(1)(2)(3)實驗後的最佳結果為表 20、21，首先從表 20 中可以得知當輸入至 3D-CNN 1 的長度為第 1~8 幀，且先做影像二值化，再標註球位置時的分類效果(表 22)最佳，分類準確度(Accuracy)高達 94.97%，至於在表 21 的部分，則是當輸入至 3D-CNN 2 的長度為第 3~11 幀，且先做影像二值化，再標註球和籃框位置時的分類效果(表 23)最佳，分類準確度(Accuracy)為 91.62%，最終將這兩組最佳的結果進行合併，並繪製混淆矩陣(表 24)來檢視模型的分類績效，分類準確度(Accuracy)從原先的 79.28%提升到 86.59%，精確度(Precision)從 80.32%提升到 86.69%，召回率(Recall)則是從原先的 78.28%提升到 86.59%，F1-score 從 79.56%提升到 86.49%，而本研究認為分類準確度(Accuracy)能再提升 7.31%的原因是，模型需要學習到的資訊量變少，再加上每個模型是個別學習投籃位置和判斷是否得分的資訊，使得預測類別變得更簡單了。因此本研究框架的 3D-CNN 架構就會分成兩個模型，第一個模型用來辨識投籃位置，第二個則是判斷是否得分，最終將這兩個模型的輸出結果合併成目標類別。

表 20 3D-CNN 1 的測試績效

輸入長度	影像二值化	標註	Accuracy	Recall	Precision	F1_score
1~7		球+籃框	88.56%	88.56%	88.72%	88.61%
1~7		球	92.18%	92.18%	92.9%	92.01%
1~6	✓	球+籃框	89.37%	89.37%	89.44%	89.41%
1~8	✓	球	94.97%	94.97%	95.03%	94.95%

表 21 3D-CNN 2 的測試績效

輸入長度	影像二值化	標註	Accuracy	Recall	Precision	F1_score
3~10		球+籃框	87.33%	87.34%	87.58%	87.46%
3~11		球	83.51%	83.51%	83.68%	83.59%
3~11	✓	球+籃框	91.62%	89.25%	94.32%	91.71%
3~9	✓	球	86.15%	86.15%	86.64%	86.37%

表 22 測試 3D-CNN 1 之混淆矩陣

總樣本數(179 組)		預測值		
		3pt	2pt	1pt
實際值	3pt	55	2	0
	2pt	6	78	0
	1pt	0	1	37

表 23 測試 3D-CNN 2 之混淆矩陣

總樣本數(179 組)		預測值	
		未得分	得分
實際值	未得分	81	10
	得分	5	83

表 24 合併 3D-CNN 1 和 3D-CNN 2 測試結果之混淆矩陣

總樣本數(179 組)		預測值					
		3pt_0	3pt_1	2pt_0	2pt_1	1pt_0	1pt_1
實際值	3pt_0	35	4	0	0	0	0
	3pt_1	1	15	0	2	0	0
	2pt_0	3	0	31	4	0	0
	2pt_1	0	3	2	41	0	0
	1pt_0	0	0	1	0	11	2
	1pt_1	0	0	0	0	2	22

5.7 將各模型預測結果合併成文字敘述的效果

建置完本研究框架後，接著會使用表 6 的測試影片，此影片約有 182 件投籃事件，可用來檢視本研究框架生成文字敘述的效果，其中如表 25 所示，資料欄位包含第幾節(Quarter)、剩餘時間(Time)、所屬球隊(Team)、球員背號(Number)和得分種類(Score Class)，而每個資料欄位都是由不同的模型進行預測以及辨識，以下會說明所有模型的預測工作，(1)OCR 辨識第幾節(Quarter)和剩餘時間(Time)，(2)透過 YOLOv5m 結合 Lightweight OpenPose，找出投籃的球員，再來辨識此球員的所屬球隊(Team)、球員背號(Number)，(3)3D-CNN 辨識得分種類(Score Class)。而本研究評估此框架生成文字敘述的方式是，只要任一資料欄位所提供的資訊是錯誤的，就會認定此框架在生成這次投籃事件的文字敘述是錯誤的，而最終生成文字敘述的效果為 81.87%，已經是本研究可以接受的範圍，但本研究也發現此框架的處理時間會比影片時間長約 40 分鐘左右，這樣在實際上線使用時，會無法達到即時播報比賽資訊的效果，因此本研究想要模擬在平行運算的情況下，需要幾台電腦才能達到即時播報的效果，這部分會在下一章節進行說明。

表 25 本研究框架生成文字敘述的效果

欄位	第幾節	剩餘時間	所屬球隊	球員背號	得分種類	整體績效
	(Quarter)	(Time)	(Team)	(Number)	(Score Class)	
準確率	98.35%	97.8%	100%	95.05%	85.71%	81.87%
命中次數	(179/182)	(178/182)	(182/182)	(173/182)	(156/182)	(149/182)

5.8 模擬平行運算所需的電腦數量

為了讓本研究框架的處理時間達到即時播報的效果，本章節模擬在平行運算的情況下所需要的電腦數量，而本研究用於測試框架的電腦設備為 Intel(R) Core(TM) i7-9700K CPU @ 3.60GHz 的 CPU、Nvidia GeForce RTX 2080 Ti 11GB 的 GPU、搭配 64GB 記憶體，總價格約 9 萬元，假設模擬的電腦設備都是一樣的，而本研究使用表 6 測試影片的 182 件投籃事件做實驗，首先會紀錄這些投籃事件所花費的時間以及電腦的處理時間，並計算平均延遲時間，再來會模擬多台電腦下的平均延遲時間，最終繪製折線圖來找出最合適的電腦數量和對應的電腦成本。以下將介紹本研究模擬電腦平行運算的過程，如表 26 所示，假設模擬 3 台電腦同時在進行 NBA 文字直播，首先可以看到第一個投籃事件需花費 21 秒完成，而電腦的處理時間為 69 秒，由於電腦 A 是閒置的狀況，因此會安排第一個投籃事件給電腦 A 處理，延遲時間是 48 秒，而第二個投籃事件需花費 10 秒，累積下來已到 31 秒，但因為電腦 A 要在 69 秒後才會閒置下來，所以會把第二個投籃事件交給電腦 B 處理，處理時間為 66 秒，延遲時間則是 35 秒，直到第四個投籃事件時，又會回來分配工作給電腦 A 處理，但電腦 A 目前是處於忙碌狀態，需要在第 69 秒後才閒置下來，而第一到第四個投籃事件才在第 52 秒位置，因此第四個投籃事件的延遲時間為前三個投籃事件所花費的時間(45 秒) + 電腦處理時間(75 秒) + 69 - 52 = 85 秒，以此類推，即可計算每個投籃事件的延遲時間，再計算平均延遲時間。最終本研究模擬 10 台電腦下的平行運算情形，結果如圖 52 所示，可以發現在電腦數量為 3 台以上時，平均延遲時間的趨勢會變得相當平緩，本研究認為較合適的電腦數量會落在 3 至 4 台，原因是當電腦數量為 3 台時，平均延遲時間為 3.33 分鐘(200.24 秒)，電腦成本為 27 萬

元，當電腦數量為 4 台時，平均延遲時間則是 1.26 分鐘(75.64 秒)，電腦成本為 36 萬元，但當電腦數量為 5 台以上時，平均延遲時間的變化會趨近於 0，電腦成本卻持續上升，對於實際上線時的效益不大，而使用者可參考模擬平行運算後的結果來決定需使用幾台電腦進行直播。

表 26 模擬電腦平行運算的過程

投籃事件	完成動作的時間 (秒)	現在時間 (秒)	電腦的處理時間 (秒)	延遲時間 (秒)	電腦 A	電腦 B	電腦 C
1	21	21	69	48	69		
2	10	31	66	35		66	
3	14	45	82	37			82
4	7	52	75	85	137		
...	...	...	...	...	...	...	...

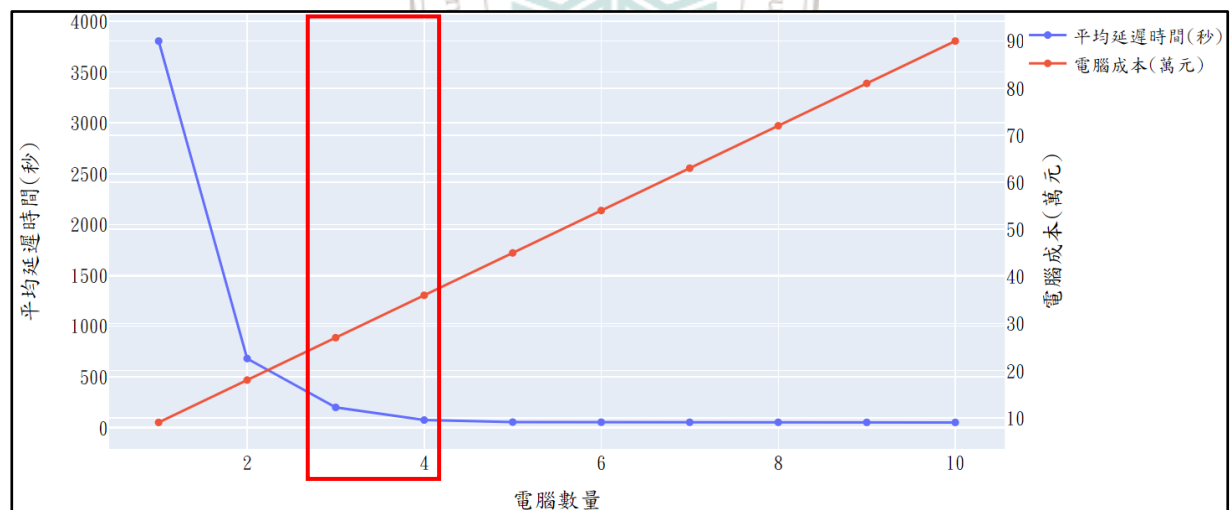


圖 52 模擬 10 台電腦平行運算後的視覺化圖

第六章 結論與未來研究建議

由於目前文字直播的工作完全透過人工處理，造成文字直播員作業上的困難點，因此本研究設計一種自動編碼器框架用於半自動化文字直播，來降低文字直播員的工作負荷量。綜觀過往研究方法應用在籃球賽事文字直播會有三大問題，(1)過往的篩選目標畫面模型必須搭配文字資料，無法應用在賽事直播情形(2)未探討投籃的球員是誰，(3)未探討不同投籃位置下的得分種類，因此本研究針對上述問題提出三大解題步驟，並分成兩個階段進行，首先階段一是串聯 CNN 和 YOLO，從多個畫面中篩選目標畫面，其篩選過程又稱為編碼器架構(Encoder)，而階段二則是使用 Lightweight OpenPose 找出投籃的球員，並透過 OCR 辨識該球員的背號，再訓練 3D-CNN 進行得分種類辨識，最終將各模型的輸出結果合併成文字敘述，也就是解碼器架構(Decoder)，建置完本研究框架後，期望未來能加入平行運算演算法，降低電腦的運行時間，來進行 NBA 文字直播。在實驗模擬的章節中，本研究透過一連串的實驗來驗證本研究框架內的模型效果，並且模擬平行運算時所需的電腦數量，最終得到以下兩個結論，(1)本研究在現階段著重在建置框架和資料分析上，其中自動編碼器框架在生成文字敘述的效果為 81.87%，期望在未來能再精進模型以提升生成文字敘述的效果。(2)經由模擬不同電腦數量下的平行運算後，本研究認為較合適的電腦數量會落在 3 至 4 台，藉以提供給使用者進行參考，期望在未來加入平行運算演算法來即時播報賽事資訊，解決延遲傳遞賽事資訊的問題，使文字直播員能透過本研究框架進行 NBA 文字直播，降低工作負荷量。

針對未來研究，我們提出五點延伸探討與建議，(1)加入 DeepSORT 演算法來追蹤場上的球員，或許能降低尋找投籃球員的時間，並且提升辨識效果。(2)測試其它物件偵測、人體姿態辨識的模型，來檢視是否能再提升辨識效果以及降低偵測的時間。(3)考慮其他動作項目如運球、傳球、抄截、阻攻、搶籃板、犯規、發生失誤等情形下的生成文字效果。(4)將此框架應用在其它運動賽事上，並檢視生成文字的效果。(5)加入平行運算演算法來降低電腦的運行時間。

參考文獻

- [1] BASKETBALL REFERENCE. from <https://www.basketball-reference.com/>
- [2] Buric, M., Pobar, M., & Ivasic-Kos, M. (2018, December). Ball detection using YOLO and Mask R-CNN. In 2018 International Conference on Computational Science and Computational Intelligence (CSCI) (pp. 319-323). IEEE.
- [3] Cao, Z., Simon, T., Wei, S. E., & Sheikh, Y. (2017). Realtime multi-person 2d pose estimation using part affinity fields. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7291-7299).
- [4] Chakma, A., Faridee, A. Z. M., Roy, N., & Hossain, H. S. (2020, March). Shoot Like Ronaldo: Predict Soccer Penalty Outcome with Wearables. In 2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops) (pp. 1-6). IEEE.
- [5] Chen, B. (2019, August 28). Object-Detection-and-Tracking. GitHub. from <https://github.com/yhengchen/Object-Detection-and-Tracking/blob/master/Two-stage%20vs%20One-stage%20Detectors.md>
- [6] Chen, C. M., & Chen, L. H. (2014, October). Novel framework for sports video analysis: a basketball case study. In 2014 IEEE International Conference on Image Processing (ICIP) (pp. 961-965). IEEE.
- [7] Choi, B., An, W., & Kang, H. (2022, October). Human Action Recognition Method using YOLO and OpenPose. In 2022 13th International Conference on Information and Communication Technology Convergence (ICTC) (pp. 1786-1788). IEEE.
- [8] COCO dataset. From <https://cocodataset.org/#home>
- [9] Deepa, R., Tamilselvan, E., Abrar, E. S., & Sampath, S. (2019, April). Comparison of yolo, ssd, faster rcnn for real time tennis ball tracking for action decision networks. In 2019 International conference on advances in computing and communication

- engineering (ICACCE) (pp. 1-4). IEEE.
- [10] Fan, W., Guo, X., Teng, L., & Wu, Y. (2021, September). Research on Abnormal Target Detection Method in Chest Radiograph Based on YOLO v5 Algorithm. In 2021 IEEE International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI) (pp. 125-128). IEEE.
- [11] Fang, H. S., Xie, S., Tai, Y. W., & Lu, C. (2017). Rmpe: Regional multi-person pose estimation. In Proceedings of the IEEE international conference on computer vision (pp. 2334-2343).
- [12] Gamra, M. B., Akhloufi, M. A., Wang, C., & Liu, S. (2021, November). Deep Learning for Body Parts Detection using HRNet and EfficientNet. In 2021 17th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS) (pp. 1-8). IEEE.
- [13] Güler, R. A., Neverova, N., & Kokkinos, I. (2018). Densepose: Dense human pose estimation in the wild. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7297-7306).
- [14] Gurung, R. (2022, September 27). Top 12 Most Popular Sports in America. PLAYERSBIO. from <https://playersbio.com/most-popular-sports-in-america/>
- [15] Hahm, G. J., & Cho, K. (2016, October). Event-based sport video segmentation using multimodal analysis. In 2016 International Conference on Information and Communication Technology Convergence (ICTC) (pp. 1119-1121). IEEE.
- [16] Hooda, P. (2021, October 27). 7 Most Popular Sports In America 2022. SPORTINGFREE. from <https://www.sportingfree.com/sports/most-popular-sports-in-america/>
<https://www.cnblogs.com/gezhuangzhuang/p/10750453.html>
- [17] Jin, H., Yan, M., Lu, J., Zhu, L., Wang, K., & Bai, S. (2019, December). Performance comparison of moving target recognition between Faster R-CNN and SSD. In 2019 International Joint Conference on Information, Media and Engineering (IJCIME) (pp. 42-46). IEEE.

- [18] Khan, M. Z., Saleem, S., Hassan, M. A., & Khan, M. U. G. (2018, November). Learning deep C3D features for soccer video event detection. In 2018 14th International Conference on Emerging Technologies (ICET) (pp. 1-6). IEEE.
- [19] Kim, J. A., Sung, J. Y., & Park, S. H. (2020, November). Comparison of Faster-RCNN, YOLO, and SSD for real-time vehicle type recognition. In 2020 IEEE international conference on consumer electronics-Asia (ICCE-Asia) (pp. 1-4). IEEE.
- [20] Lee, J., Lee, J., Moon, S., Nam, D., & Yoo, W. (2018, February). Basketball event recognition technique using Deterministic Finite Automata (DFA). In 2018 20th International Conference on Advanced Communication Technology (ICACT) (pp. 675-678). IEEE.
- [21] Li, X., & Cheng, S. (2019, December). Pedestrian gender detection based on mask r-cnn. In 2019 IEEE 5th International Conference on Computer and Communications (ICCC) (pp. 2082-2086). IEEE.
- [22] Liu, B., Zhao, W., & Sun, Q. (2017, October). Study of object detection based on Faster R-CNN. In 2017 Chinese Automation Congress (CAC) (pp. 6233-6236). IEEE.
- [23] Lv, Y. (2022, May). Research on Underground Personnel Behavior Detection Algorithm Based on Lightweight OpenPose. In 2022 5th International Conference on Artificial Intelligence and Big Data (ICAIBD) (pp. 332-336). IEEE.
- [24] NBA. From <https://www.nba.com/players>
- [25] Osokin, D. (2018). Real-time 2d multi-person pose estimation on cpu: Lightweight openpose. arXiv preprint arXiv:1811.12004.
- [26] P. Kumar Doddamani & G. P. Revathi. (2022). Detection of Weed & Crop using YOLO v5 Algorithm. In 2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon) (pp. 1-5). IEEE.
- [27] Park, C., Lee, H. S., Kim, W. J., Bae, H. B., Lee, J., & Lee, S. (2021). An Efficient Approach Using Knowledge Distillation Methods to Stabilize Performance in a

- Lightweight Top-Down Posture Estimation Network. *Sensors*, 21(22), 7640.
- [28] Park, J. H., & Cho, K. (2016, October). Extraction of visual information in basketball broadcasting video for event segmentation system. In 2016 International Conference on Information and Communication Technology Convergence (ICTC) (pp. 1098-1100). IEEE.
- [29] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 779-788).
- [30] Sen, A., Hossain, S. M. M., Russo, M. A., Deb, K., & Jo, K. H. (2022, July). Fine-Grained Soccer Actions Classification Using Deep Neural Network. In 2022 15th International Conference on Human System Interaction (HSI) (pp. 1-6). IEEE.
- [31] Shakya, S. R., Zhang, C., & Zhou, Z. (2021, May). Basketball-51: A Video Dataset for Activity Recognition in the Basketball Game. In CS & IT Conference Proceedings (Vol. 11, No. 7). CS & IT Conference Proceedings.
- [32] Shi, K., Bao, H., & Ma, N. (2017, December). Forward vehicle detection based on incremental learning and fast R-CNN. In 2017 13th International Conference on Computational Intelligence and Security (CIS) (pp. 73-76). IEEE.
- [33] Sun, K., Xiao, B., Liu, D., & Wang, J. (2019). Deep high-resolution representation learning for human pose estimation. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 5693-5703).
- [34] Sun, R., Zhang, Q., Luo, C., Guo, J., & Chai, H. (2022). Human action recognition using a convolutional neural network based on skeleton heatmaps from two-stage pose estimation. *Biomimetic Intelligence and Robotics*, 2(3), 100062.
- [35] Tegicho, B. E., Chestnut, M. D., Webb, D., & Graves, C. (2021, December). Basketball Shot Analysis Based on Goal Assembly Disturbance. In 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference

- (UEMCON) (pp. 0498-0503). IEEE.
- [36] Verma, S., Tripathi, S., Singh, A., Ojha, M., & Saxena, R. R. (2021, December). Insect Detection and Identification using YOLO Algorithms on Soybean Crop. In TENCON 2021-2021 IEEE Region 10 Conference (TENCON) (pp. 272-277). IEEE.
- [37] Wu, L., Yang, Z., He, J., Jian, M., Xu, Y., Xu, D., & Chen, C. W. (2019). Ontology-based global and collective motion patterns for event classification in basketball videos. IEEE Transactions on Circuits and Systems for Video Technology, 30(7), 2178-2190.
- [38] Yan, C., Li, X., & Li, G. (2021, August). A new action recognition framework for video highlights summarization in sporting events. In 2021 16th International Conference on Computer Science & Education (ICCSE) (pp. 653-666). IEEE.
- [39] Yang, Q., Shao, J., & Zuo, H. (2021, June). Automatic Analysis of Basketball Shooting Based on Machine Learning. In 2021 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA) (pp. 1233-1238). IEEE.
- [40] YouTube.[Video] from <https://www.youtube.com/>
- [41] Zhang, H., Du, Y., Ning, S., Zhang, Y., Yang, S., & Du, C. (2017, December). Pedestrian detection method based on Faster R-CNN. In 2017 13th International Conference on Computational Intelligence and Security (CIS) (pp. 427-430). IEEE.
- [42] Zhang, Y., Chen, Z., & Wei, B. (2020, December). A sport athlete object tracking based on deep sort and yolo V4 in case of camera movement. In 2020 IEEE 6th International Conference on Computer and Communications (ICCC) (pp. 1312-1316). IEEE.
- [43] 江浙滬唯一貧困人口（民 109 年）。虎撲 APP 產品分析：JR 的聚集地，一個認真有趣的社區。人人都是產品經理。取自 <https://www.woshipm.com/evaluating/3278945.html>
- [44] 角點偵測示意圖。取自 <https://www.cnblogs.com/gezhuangzhuang/p/10750453.html>
- [45] 胡圖圖（民 109 年）。騰訊體育、虎撲體育籃球模塊競品分析報告。知乎。取自

附件一 NBA 比賽的記分板位置

附表 1 本研究記錄 44 場 NBA 比賽計分板位置(2010~2016 年)

影片編號	對戰隊伍和時間	記分板位置	記分板(Y)	記分板(X)
1	太陽 VS 湖人(2010.05.27)	右下方	520~650	780~1080
2	雷霆 VS 金塊(2012.02.19)	正下方	570~650	310~970
3	尼克 VS 公牛(2012.04.08)	正下方	570~650	280~920
4	雷霆 VS 湖人(2012.05.19)	右下方	560~640	910~1200
5	賽爾蒂克 VS 熱火(2012.05.30)	正下方	570~650	250~1030
6	熱火 VS 湖人(2013.01.17)	右下方	570~630	810~1200
7	勇士 VS 尼克(2013.02.27)	正下方	570~650	310~970
8	雷霆 VS 火箭(2013.04.21)	右下方	540~640	810~1200
9	馬刺 VS 勇士(2013.05.06)	右下方	570~630	810~1200
10	溜馬 VS 熱火(2013.05.24)	右下方	550~640	820~1210
11	馬刺 VS 熱火(2013.06.19)	正下方	570~650	280~910
12	熱火 VS 籃網(2014.05.14)	右下方	540~640	810~1200
13	快艇 VS 馬刺(2015.05.02)	右下方	570~640	800~1200
14	騎士 VS 公牛(2015.05.08)	正下方	560~650	280~1000
15	火箭 VS 勇士(2015.05.21)	正下方	560~650	280~990
16	勇士 VS 騎士(2015.12.25)	正下方	560~650	280~990
17	勇士 VS 騎士(2016.01.18)	右下方	570~630	810~1200
18	勇士 VS 雷霆(2016.02.27)	正下方	560~650	290~1000
19	騎士 VS 爵士(2016.03.14)	正下方	560~620	270~990
20	騎士 VS 老鷹(2016.05.06)	正下方	560~650	280~1000
21	暴龍 VS 熱火(2016.05.07)	正下方	560~650	290~990
22	勇士 VS 拓荒者(2016.05.11)	右下方	570~630	810~1200
23	尼克 VS 勇士(2016.12.15)	右下方	540~650	1000~1220

附表 2 本研究記錄 44 場 NBA 比賽計分板位置(2017~2023 年)

影片編號	對戰隊伍和時間	記分板位置	記分板(Y)	記分板(X)
26	騎士 VS 鵜鶘(2017.01.02)	正下方	590~630	190~1080
27	雷霆 VS 公鹿(2017.04.04)	正下方	630~670	100~1180
28	雷霆 VS 爵士(2018.04.25)	正下方	600~640	220~1050
29	暴龍 VS 騎士(2018.05.05)	正下方	630~680	110~1160
30	騎士 VS 勇士(2018.06.06)	正下方	640~690	210~1050
31	拓荒者 VS 金塊(2019.05.12)	正下方	600~680	300~1000
32	熱火 VS 湖人(2020.10.11)	正下方	590~690	260~1010
33	勇士 VS 熱火(2021.02.17)	正下方	630~720	0~1280
34	公鹿 VS 馬刺(2021.10.23)	正下方	650~700	0~1280
35	獨行俠 VS 76 人(2022.02.04)	正下方	640~700	20~1260
36	國王 VS 雷霆(2022.02.05)	右下方	540~650	1000~1220
37	公鹿 VS 快艇(2022.02.06)	正下方	650~700	0~1280
38	火箭 VS 鵜鶘(2022.02.08)	正下方	650~700	0~1280
39	馬刺 VS 雷霆(2022.02.16)	正下方	650~700	0~1280
40	湖人 VS 勇士(2022.03.05)	正下方	630~700	20~1260
41	爵士 VS 雷霆(2022.03.06)	右下方	610~690	830~1190
42	湖人 VS 公鹿(2023.02.09)	右下方	590~660	940~1220
43	賽爾蒂克 VS 黃蜂(2023.02.10)	正下方	630~700	300~980
44	獨行俠 VS 國王(2023.02.10)	右下方	550~650	1000~1220

附表 3 角點偵測的範圍

記分板位置	記分板(Y)	記分板(X)
正下方	540~700	460~1080
右下方	520~690	780~1220

